

Agentic Signal versus Background Classification for Exotic Higgs Decays

This tutorial uses a collider-physics binary classification problem. The goal is to distinguish an exotic-Higgs signal from Standard Model heavy-flavour background.

Signal process	Background process
$pp \rightarrow H$	$pp \rightarrow$ QCD heavy flavour
$H \rightarrow aa$	examples: $b\bar{b}j$, $b\bar{b}jj$, $b\bar{b}b\bar{b}$
$a \rightarrow b\bar{b}$ and $a \rightarrow b\bar{b}$	multijet final states with heavy-flavour jets
final state: $b\bar{b}b\bar{b}$	
	final state: jets that can mimic four-b signal events

Signal Feynman-diagram sketch	Background Feynman-diagram sketch
$pp \rightarrow H \rightarrow aa \rightarrow (b\bar{b})(b\bar{b})$	$pp \rightarrow$ QCD $\rightarrow b\bar{b} +$ jets

The machine-learning task is binary classification:

- `label = 1` : exotic-Higgs signal.
- `label = 0` : QCD heavy-flavour background.

The physics objective is to identify events that are more signal-like than background-like, then check whether applying a classifier threshold improves a simple counting sensitivity. The workflow uses `Avencast/EveNet-ExoticHiggs-H2a4b`, associated with arXiv:2601.17126 and released with an MIT license. Only a small number of Parquet files are downloaded; the full dataset is not required.

Section 1 - Setup Data

The helper `load_physics_pipeline` handles the low-level parsing of the EveNet Parquet file and returns a compact pandas dataframe called `selected_events`. The dataframe is arranged so the physics objects have consistent names:

- `label` : signal/background class label.
- `jet1`, `jet2`, `jet3`, `jet4` : the four leading jets ordered by transverse momentum.
- `pt` : transverse momentum, the momentum perpendicular to the proton beam.
- `eta` : pseudorapidity, a coordinate related to the jet direction along the beam axis.
- `phi` : azimuthal angle around the beam axis.
- `mass` : reconstructed jet mass.
- `btag` : b-tag information used to identify jets likely produced by bottom quarks.

The initial plots below show the cleaned four-jet sample before any agent modifies features or classifiers.

Infrastructure note: helper functions hide low-level file parsing and LLM response validation. Model training code is intentionally visible in the notebook through one `ClassificationAgent` class with a small model library.

Visible helper contracts:

Helper	Input	Output
<code>load_physics_pipeline(...)</code>	number of files, event cap, random seed	dictionary containing <code>selected_events</code> , cut-flow tables, and schema summaries
<code>request_strategy_from_cerebras(...)</code>	prompt, history, dataset summary, baseline summary, agent catalog	validated strategy dictionary
<code>summarize_planning_round(...)</code>	round number, plan, classifier agent, best threshold result	compact round summary for the next planning prompt

```
In [29]: %matplotlib inline

import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

from sklearn.ensemble import ExtraTreesClassifier, HistGradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    RocCurveDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Use paths relative to the notebook location so this folder can be copied elsewhere.
notebook_dir = Path.cwd()
if not (notebook_dir / "agentic_lab7").exists() and (notebook_dir / "lab7" / "agentic_lab7").exists():
    notebook_dir = notebook_dir / "lab7"
if str(notebook_dir) not in sys.path:
    sys.path.insert(0, str(notebook_dir))

from agentic_lab7.api_key import CEREBRAS_API_KEY, CEREBRAS_MODEL
from agentic_lab7.pipeline_setup import load_physics_pipeline
from agentic_lab7.planning_tools import request_strategy_from_cerebras, summarize_round as summarize_planning_round

RANDOM_STATE = 123
MAX_EVENTS = 200_000
np.random.seed(RANDOM_STATE)
torch.manual_seed(RANDOM_STATE)
plt.rcParams["figure.figsize"] = (7, 5)
```

```
In [30]: # Number of data files to use for loading events
N_FILES_TO_USE = 24
# Load physics data using the helper pipeline
pipeline = load_physics_pipeline(
    n_files_to_use=N_FILES_TO_USE, # Specify how many files to read
    max_events=MAX_EVENTS, # Specify the maximum number of events to load
    random_state=RANDOM_STATE, # Ensure reproducibility
)

# Extract the selected events DataFrame from the pipeline output
selected_events = pipeline["selected_events"]

# Display a summary of the loaded data, including raw events, selected events, and number of columns
display(pd.DataFrame({
    "raw_events_loaded": [len(pipeline["data"])],
    "selected_events": [len(selected_events)],
    "columns": [selected_events.shape[1]],
}))
```

```
# Display the first 5 rows of the selected events DataFrame
display(selected_events.head())

# Engineer a new feature: 'n_btags_raw' - the sum of b-tag values for the four jets
# This counts how many jets are b-tagged (assuming btag values are 0 or 1)
selected_events["n_btags_raw"] = selected_events[["jet{i}_btag" for i in range(1, 5)]].sum(axis=1)

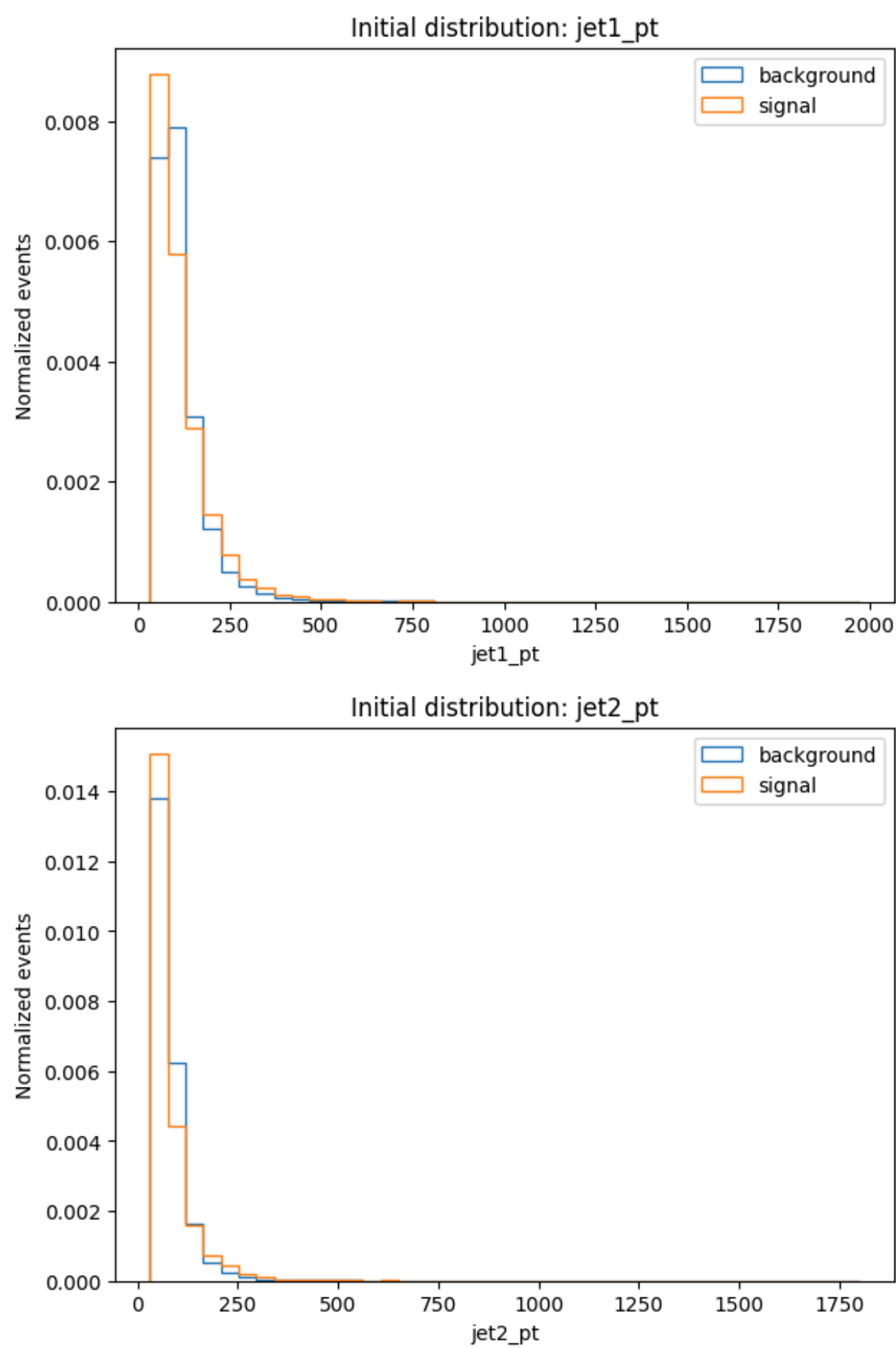
# Define columns to plot initial distributions for
plot_columns = ["jet1_pt", "jet2_pt", "jet3_pt", "jet4_pt", "n_btags_raw"]

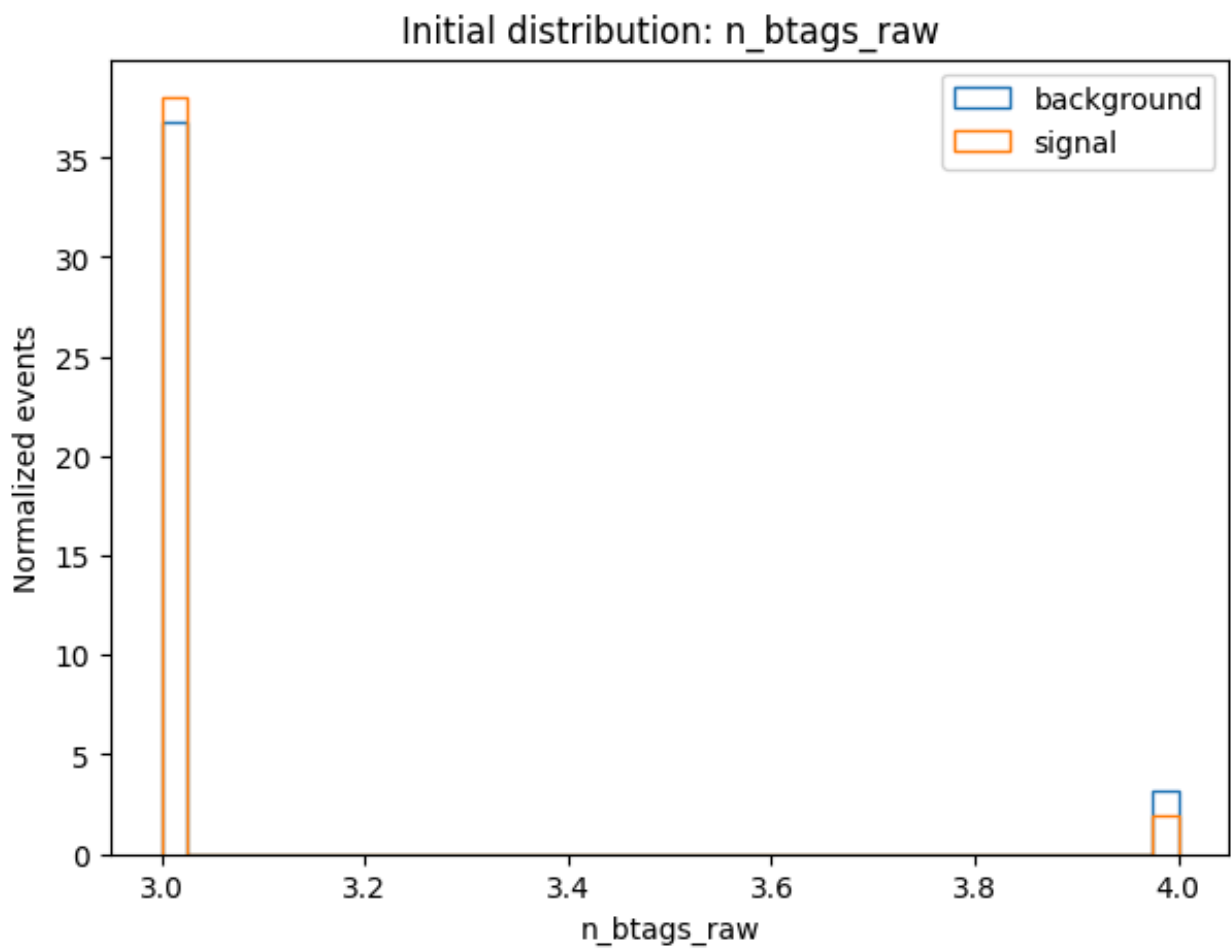
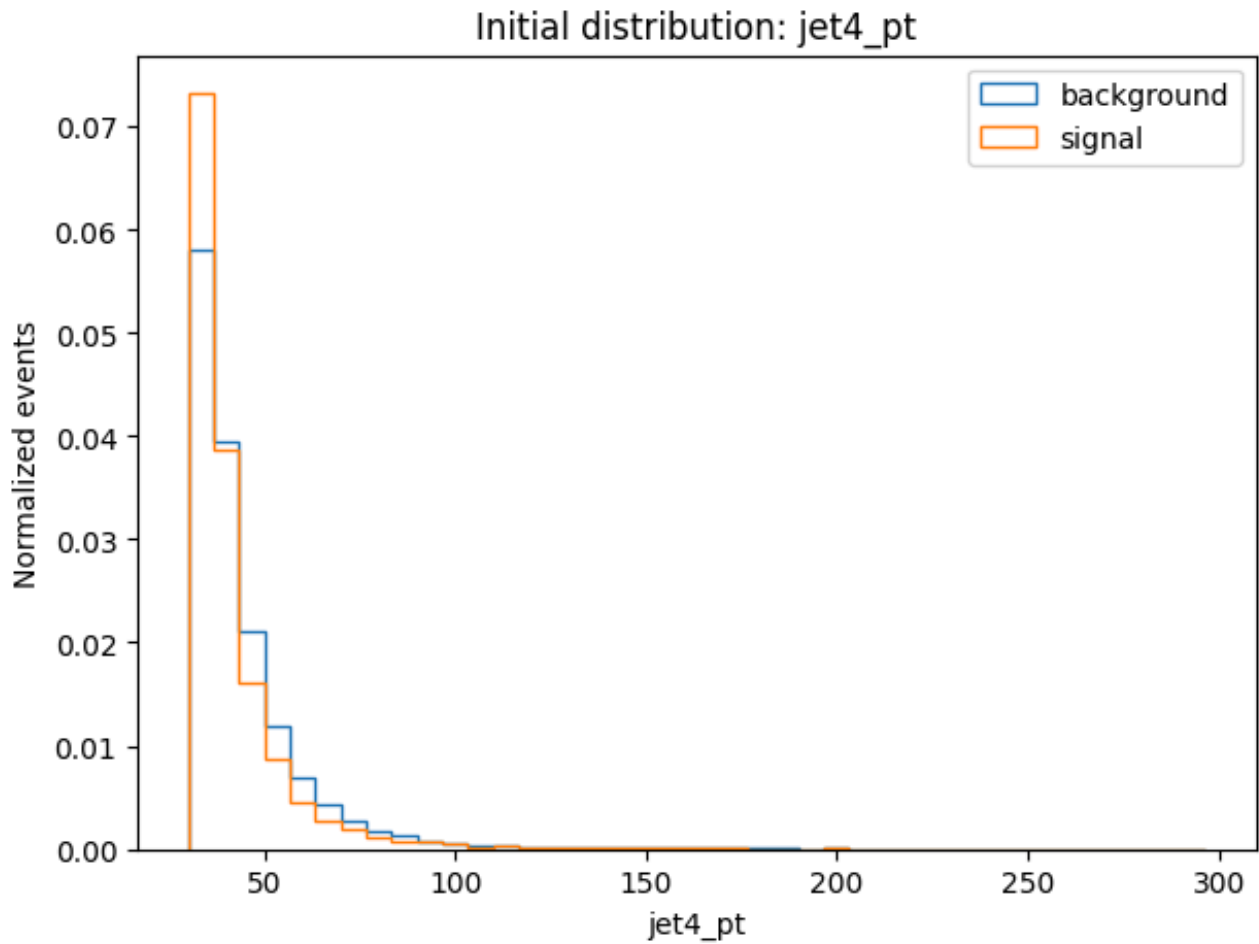
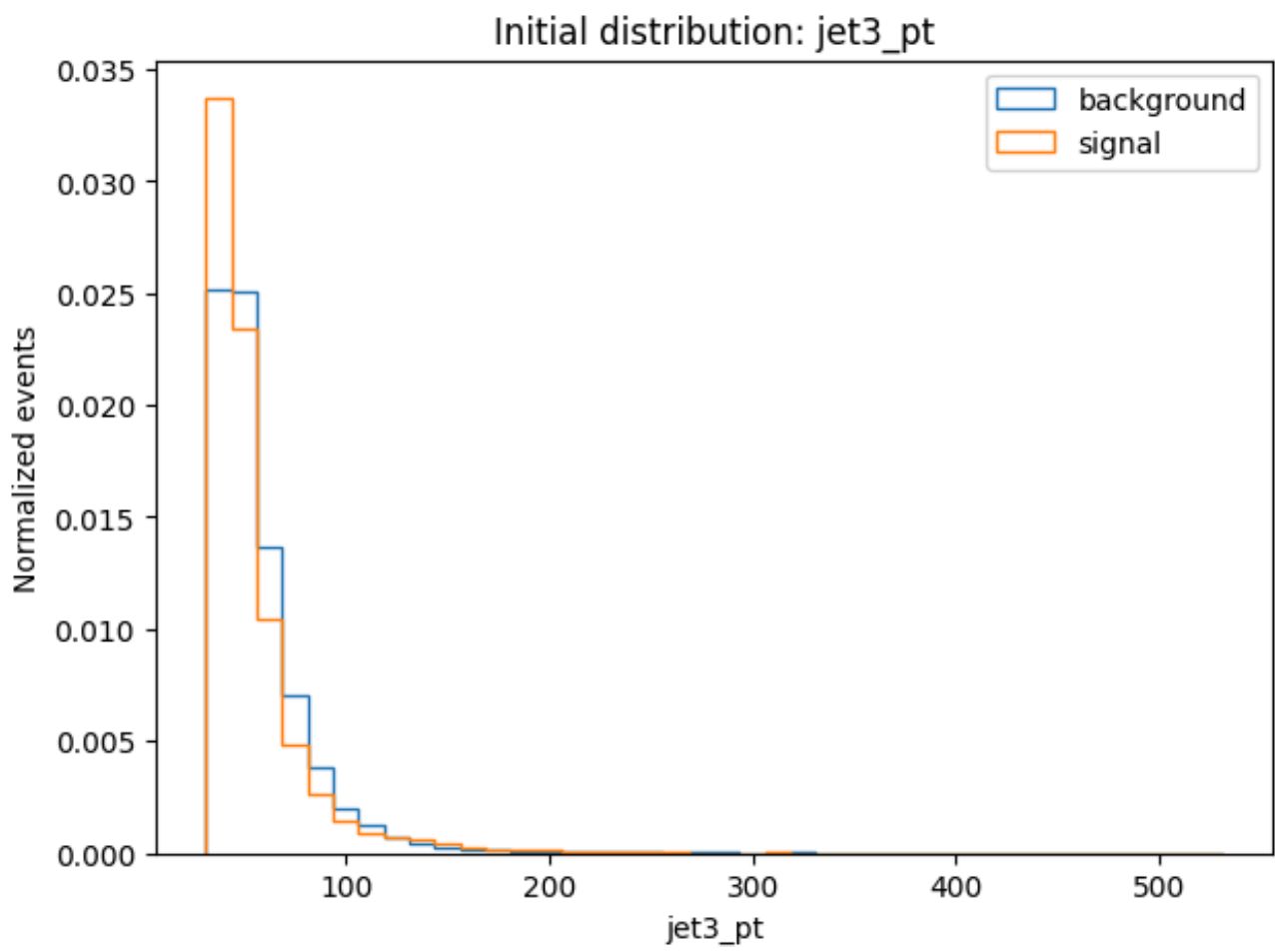
# Loop through each column and plot its distribution for signal and background events
for column in plot_columns:
    # Get non-NaN values for the current column
    values = selected_events[column].dropna().to_numpy()
    # Determine bin edges for the histogram
    bins = np.histogram_bin_edges(values, bins=40)
    plt.figure() # Create a new figure for each plot
    # Plot histogram for background events (label == 0)
    for label, name in [(0, "background"), (1, "signal")]:
        class_values = selected_events.loc[selected_events["label"] == label, column].dropna().to_numpy()
        plt.hist(class_values, bins=bins, histtype="step", density=True, label=name) # 'step' for outlines, 'density' for normalized histograms
    plt.xlabel(column) # Set x-axis label
    plt.ylabel("Normalized events") # Set y-axis label
    plt.title(f"Initial distribution: {column}") # Set plot title
    plt.legend() # Display legend
    plt.show() # Show the plot
```

	raw_events_loaded	selected_events	columns
0	200000	75126	21

	label	jet1_pt	jet1_eta	jet1_phi	jet1_mass	jet1_btag	jet2_pt	jet2_eta	jet2_phi	jet2_mass	...	jet3_pt	jet3_eta	jet3_phi	jet3_mass	jet3_btag	jet4_pt	jet4_eta	jet4_phi	j
0	1.0	93.664531	-0.087359	-2.475345	9.789335	1.0	80.338509	1.576945	0.333819	0.000000	...	34.622795	0.511399	-2.549871	5.108600	1.0	32.414450	0.552133	2.214729	
1	1.0	61.916652	0.902399	0.266193	0.000000	0.0	47.088154	0.198319	2.316577	8.675580	...	46.221401	0.692270	-2.392012	1.113719	1.0	37.838821	0.947641	2.469884	
2	1.0	67.839815	-0.506841	-1.267455	8.586977	1.0	62.954586	-2.280551	-0.061890	0.000000	...	50.028944	-2.729800	1.251373	0.000000	1.0	44.099123	-3.184988	-2.491740	
3	0.0	52.155311	0.724185	1.833906	1.579675	1.0	46.139782	-2.184562	-1.162588	0.000000	...	39.789092	-0.756486	0.927674	2.875871	1.0	38.916300	-0.945284	-1.449909	
4	0.0	73.618286	0.643026	1.586173	5.096674	1.0	70.661523	-0.555229	-2.147839	10.997287	...	67.678974	0.281106	-2.774277	9.377133	1.0	60.258595	-1.421532	0.500439	

5 rows × 21 columns





Section 2 - Feature Agent Template Class

The feature agent transforms the cleaned event dataframe into an ML-ready feature matrix. It keeps the direct four-jet measurements and adds derived physics variables that summarize the event topology.

Direct per-jet features:

- `jetN_pt` : transverse momentum of jet `N` .
- `jetN_eta` : pseudorapidity of jet `N` .
- `jetN_phi` : azimuthal angle of jet `N` .
- `jetN_mass` : reconstructed mass of jet `N` .
- `jetN_btag` : b-tag value for jet `N` ; the code counts a jet as tagged when this value is at least 0.5.

Derived event features:

- `ht` : scalar sum of the four jet transverse momenta, $p_{T,1} + p_{T,2} + p_{T,3} + p_{T,4}$.
- `m_4j` : invariant mass of the four-jet system.
- `m_jj_12` , ..., `m_jj_34` : invariant masses of all six jet pairs.
- `deltaR_jj_12` , ..., `deltaR_jj_34` : angular separations between jet pairs, using $\Delta R = \sqrt{(\Delta\eta)^2 + (\Delta\phi)^2}$ with $\Delta\phi$ wrapped into $[-\pi, \pi]$.
- `min_deltaR_jj` : smallest pairwise jet separation in the event.
- `mean_deltaR_jj` : average pairwise jet separation in the event.
- `n_btags` : number of the four leading jets passing the b-tag threshold.

These variables encode the signal hypothesis. In *Hoaaobarbbarb*, the final state contains four b quarks, often organized into two pairs from the intermediate particles `a` . QCD background can also contain b jets, but its pair masses, angular structure, total activity, and b-tag multiplicity do not have to follow the same pattern.

```
In [33]: # Fill in the feature construction inside FeatureAgent.
class FeatureAgent:
    """A class to transform raw event data into an ML-ready feature matrix."""

    def __init__(self, name="four_jet_feature_agent"):
        self.name = name # Name of the feature agent
        self.X = None # Feature matrix
        self.y = None # Target labels

    @staticmethod
```

```

def delta_phi(phi1, phi2):
    """
    Calculates the delta (difference) in azimuthal angle, handling periodicity.
    """
    delta_phi = np.arctan2(np.sin(phi2 - phi1), np.cos(phi2 - phi1))
    return delta_phi

@staticmethod
def four_vector(df, jet):
    """
    Constructs the four-vector components (energy, px, py, pz) for a given jet.
    """
    # Extract relevant jet features, ensuring float type
    pt = df[f"jet{jet}_pt"].astype(float)
    eta = df[f"jet{jet}_eta"].astype(float)
    phi = df[f"jet{jet}_phi"].astype(float)
    mass = df[f"jet{jet}_mass"].astype(float)

    # YOUR CODE HERE: Calculate px, py, pz, and energy from pt, eta, phi, and mass.
    px = pt*np.cos(phi)
    py = pt*np.sin(phi)
    pz = pt*np.sinh(eta)
    energy = np.sqrt(mass**2 + pt**2*np.cosh(eta)**2)

    return pd.DataFrame({"E": energy, "px": px, "py": py, "pz": pz}, index=df.index)

@staticmethod
def invariant_mass(vectors):
    """Calculates the invariant mass from four-vector components.
    """
    # Calculate the square of the 3-momentum (px^2 + py^2 + pz^2)
    p2 = vectors["px"]**2 + vectors["py"]**2 + vectors["pz"]**2
    # Invariant mass squared (m^2) = E^2 - p^2
    # m = sqrt(E^2 - p^2)
    m = np.sqrt(vectors["E"]**2 - (p2))
    return m

def delta_r(self, df, i, j):
    """Calculates the angular separation Delta R between two jets.
    """
    # Calculate difference in pseudorapidity
    deta = df[f"jet{i}_eta"] - df[f"jet{j}_eta"]
    # Calculate difference in azimuthal angle using the static method
    dphi = self.delta_phi(df[f"jet{i}_phi"], df[f"jet{j}_phi"])
    # Delta R = sqrt(deta^2 + dphi^2)
    delta_r = np.sqrt(deta**2 + dphi**2)
    return delta_r

def features(self, selected_events):
    """Engineers features from the selected_events DataFrame.
    """
    df = selected_events.copy() # Work on a copy to avoid modifying original data
    X = pd.DataFrame(index=df.index) # Initialize an empty DataFrame for features

    # Direct jet features: these are copied directly.
    for jet in range(1, 5):
        for field in ["pt", "eta", "phi", "mass", "btag"]:
            X[f"jet{jet}_{field}"] = df[f"jet{jet}_{field}"].astype(float)

    # Create four-vectors for each jet
    vectors = {jet: self.four_vector(df, jet) for jet in range(1, 5)}

    # YOUR CODE HERE: Calculate 'ht' (scalar sum of transverse momenta)
    X["ht"] = df["jet1_pt"] + df["jet2_pt"] + df["jet3_pt"] + df["jet4_pt"]

    # YOUR CODE HERE: Calculate 'm_4j' (invariant mass of the four-jet system)
    # Sum the four-vectors of all jets
    total = vectors[1] + vectors[2] + vectors[3] + vectors[4]
    # Calculate the invariant mass of the total four-vector
    X["m_4j"] = self.invariant_mass(total)

    delta_r_values = [] # List to store Delta R values for later min/mean calculation
    # Loop through all unique jet pair
    for i in range(1, 5):
        for j in range(i + 1, 5):
            # YOUR CODE HERE: Calculate invariant mass for each jet pair (m_jj_ij)
            # Sum the four-vectors of the current pair
            pair = vectors[i] + vectors[j]
            # Calculate the invariant mass of the pair
            X[f"m_jj_{i}{j}"] = self.invariant_mass(pair)

            # YOUR CODE HERE: Calculate angular separation Delta R for each jet pair (deltaR_jj_ij)
            X[f"deltaR_jj_{i}{j}"] = self.delta_r(df, i, j)
            delta_r_values.append(X[f"deltaR_jj_{i}{j}"]) # Add to list for min/mean

    # Concatenate all delta_r_values into a single DataFrame for easier row-wise operations
    delta_r_table = pd.concat(delta_r_values, axis=1)

    # YOUR CODE HERE: Calculate 'min_deltaR_jj'
    X["min_deltaR_jj"] = delta_r_table.min(axis=1)

    # YOUR CODE HERE: Calculate 'mean_deltaR_jj'
    X["mean_deltaR_jj"] = delta_r_table.mean(axis=1)

    # YOUR CODE HERE: Calculate 'n_btags'
    X["n_btags"] = (df["jet1_btag"].astype(float) >= 0.5).astype(int) \
        + (df["jet2_btag"].astype(float) >= 0.5).astype(int) \
        + (df["jet3_btag"].astype(float) >= 0.5).astype(int) \
        + (df["jet4_btag"].astype(float) >= 0.5).astype(int)

    # Extract the 'label' column as the target variable 'y'
    y = df["label"].astype(int)

    # Assert that the feature matrix does not contain any NaN or infinite values after feature engineering
    assert np.isfinite(X.to_numpy()).all()
    self.X = X # Store the feature matrix
    self.y = y # Store the labels
    return X, y

def describe(self):
    """Returns a string describing the current state of the FeatureAgent."""
    n_events = len(self.X) if self.X is not None else 0
    n_features = self.X.shape[1] if self.X is not None else 0
    return f"FeatureAgent `{self.name}` built {n_features} ML features for {n_events:,} selected events."

# Create an instance of FeatureAgent
feature_agent = FeatureAgent()
# Generate features and labels from the selected events
X, y = feature_agent.features(selected_events)
# Print a description of the generated features
print(feature_agent.describe())
# Display the first 5 rows of the feature matrix
display(X.head())
# Display the count of each class in the labels (0 and 1)
display(y.value_counts().sort_index().rename("events"))

```

FeatureAgent `four\_jet\_feature\_agent` built 37 ML features for 75,126 selected events.

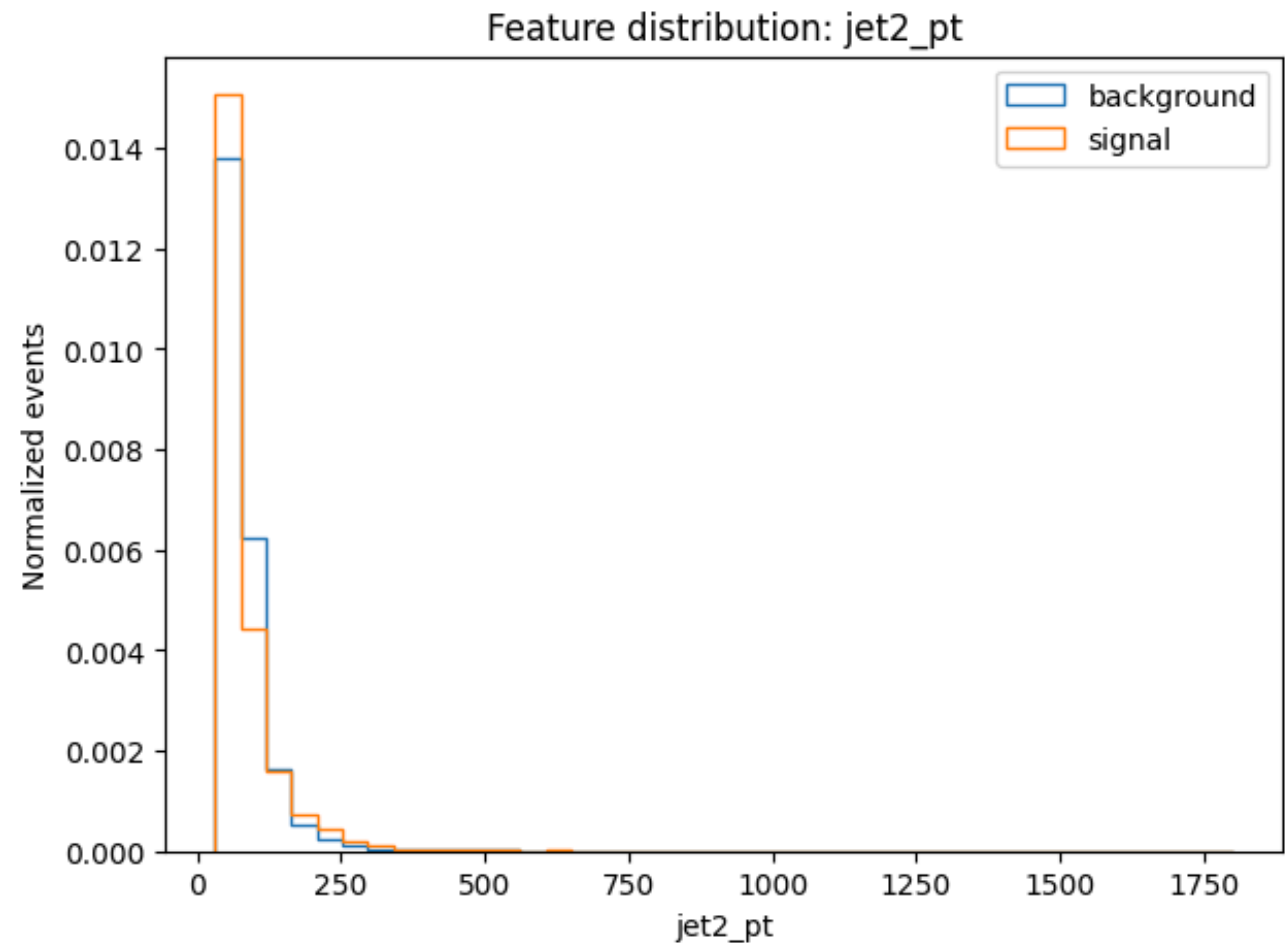
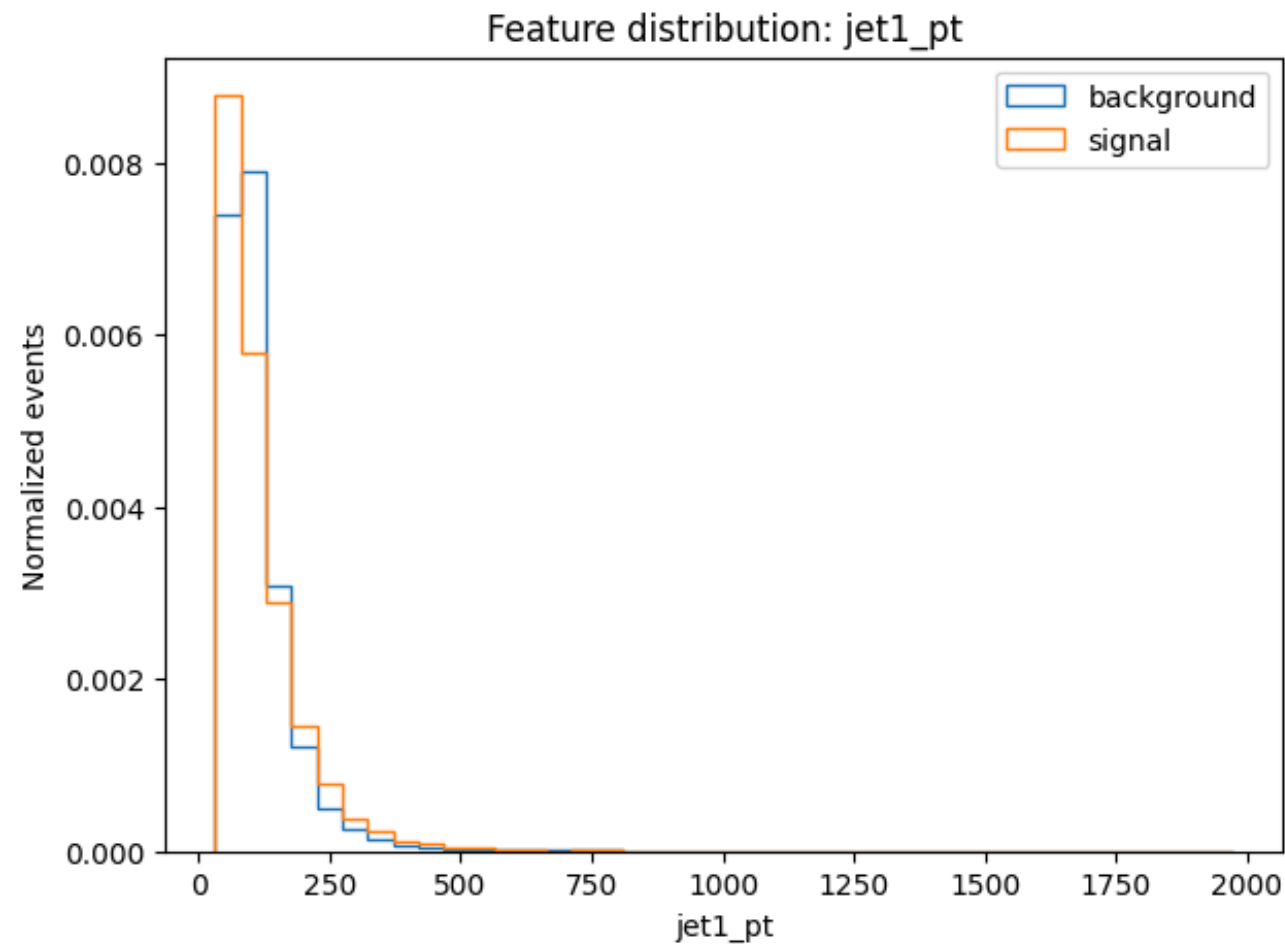
	jet1_pt	jet1_eta	jet1_phi	jet1_mass	jet1_btag	jet2_pt	jet2_eta	jet2_phi	jet2_mass	jet2_btag	...	deltaR_jj_14	m_jj_23	deltaR_jj_23	m_jj_24	deltaR_jj_24	m_jj_34	deltaR_jj_34
0	93.664531	-0.087359	-2.475345	9.789335	1.0	80.338509	1.576945	0.333819	0.000000	0.0	...	1.716670	120.710389	3.074257	99.099525	2.141976	46.830577	1.519
1	61.916652	0.902399	0.266193	0.000000	0.0	47.088154	0.198319	2.316577	8.675580	1.0	...	2.204155	71.265440	1.650255	35.390279	0.764843	55.624029	1.444
2	67.839815	-0.506841	-1.267455	8.586977	1.0	62.954586	-2.280551	-0.061890	0.000000	1.0	...	2.944715	73.083284	1.387979	110.394894	2.592716	92.279513	2.580
3	52.155311	0.724185	1.833906	1.579675	1.0	46.139782	-2.184562	-1.162588	0.000000	1.0	...	3.432688	99.805236	2.531520	59.121267	1.272149	74.137131	2.385
4	73.618286	0.643026	1.586173	5.096674	1.0	70.661523	-0.555229	-2.147839	10.997287	1.0	...	2.332642	75.998871	1.044931	140.471465	2.786370	177.377298	3.456

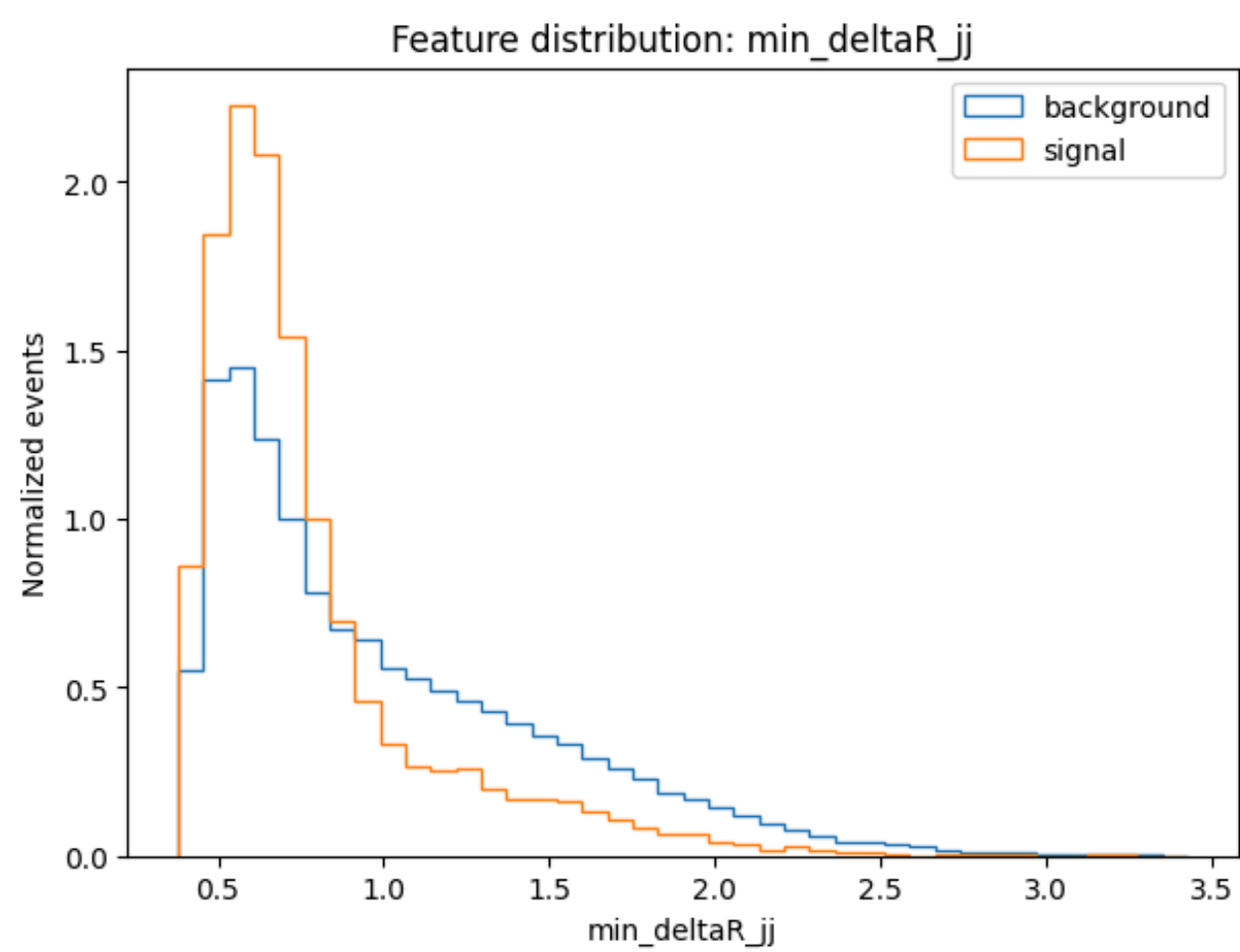
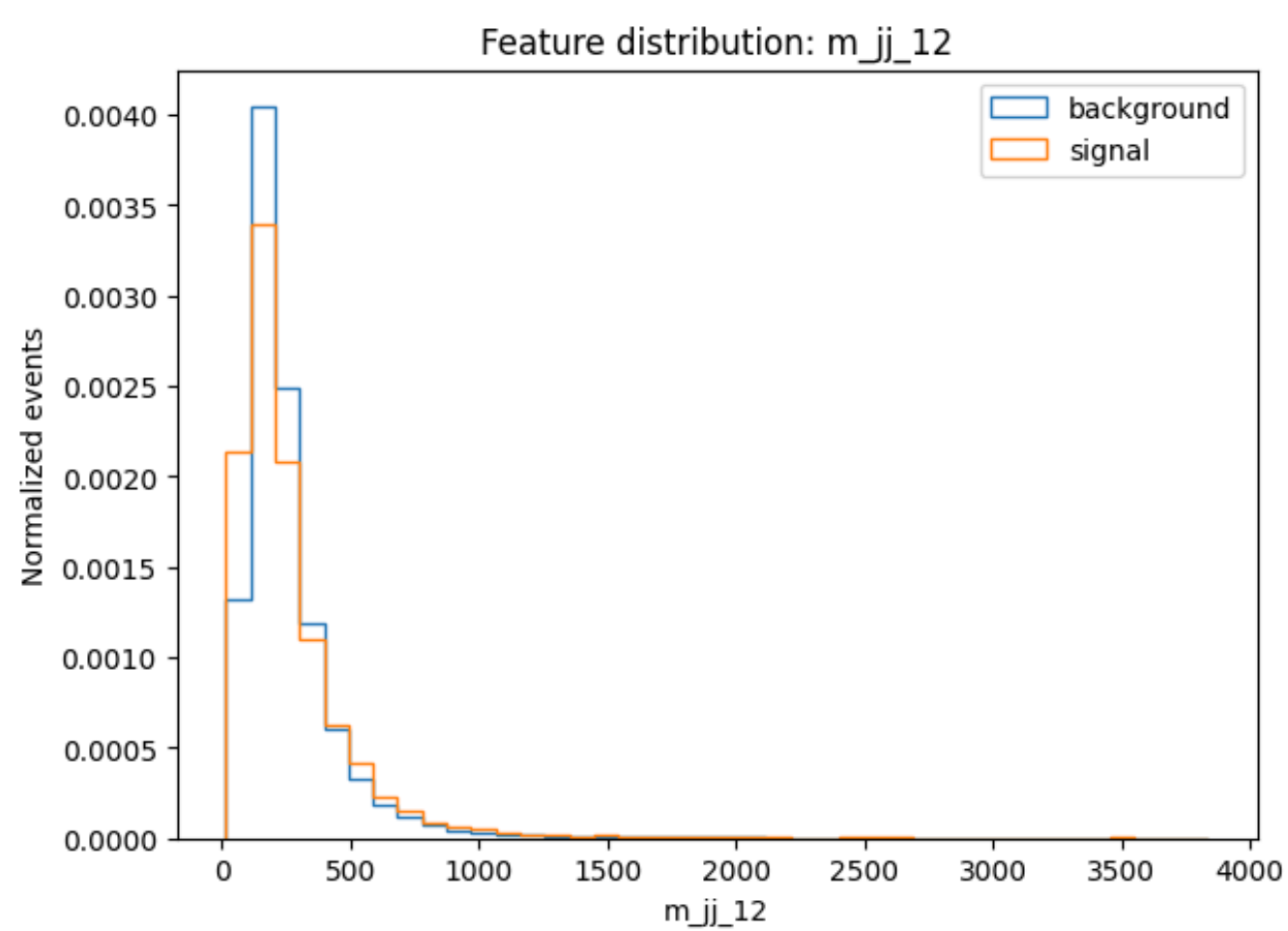
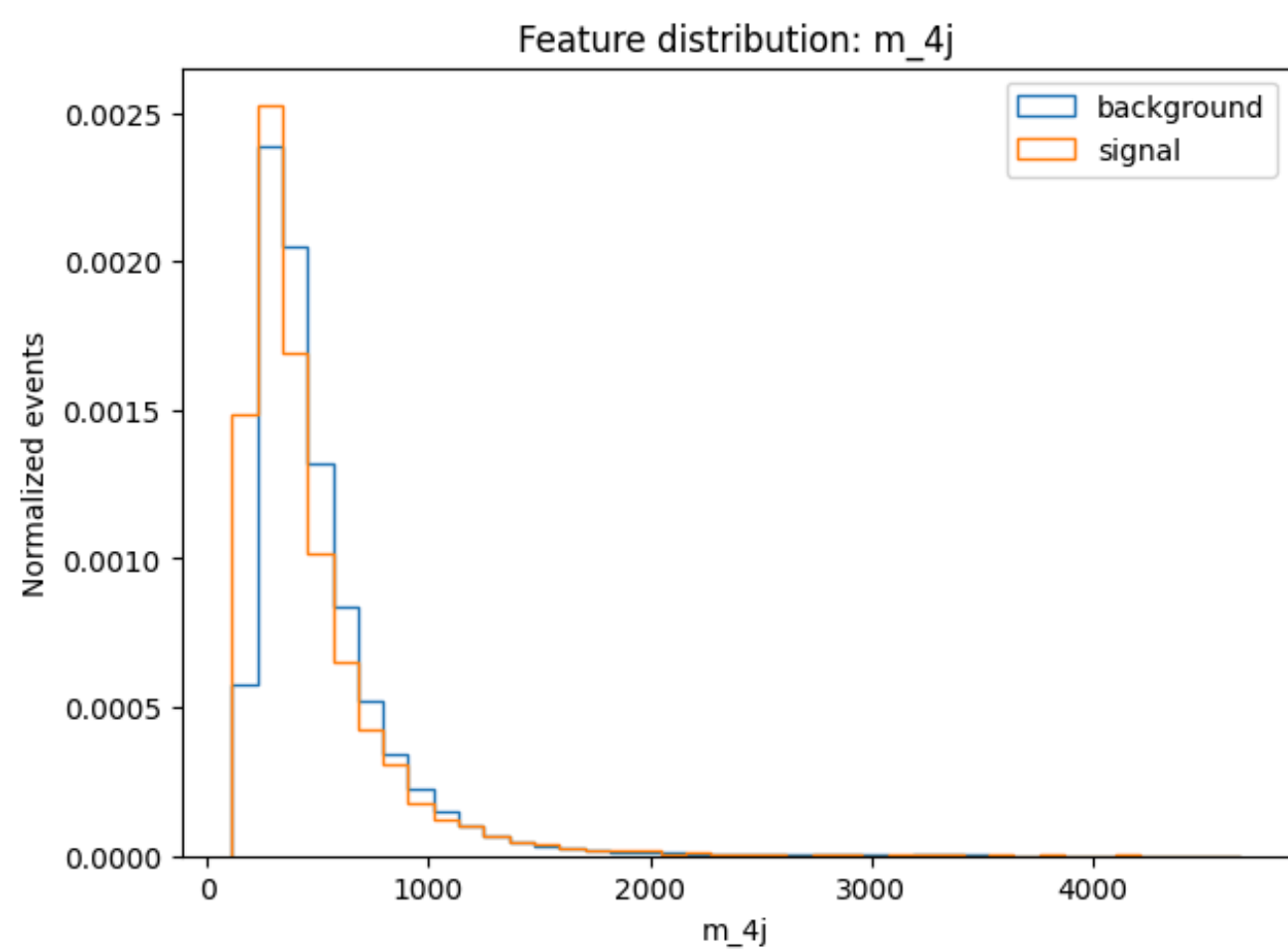
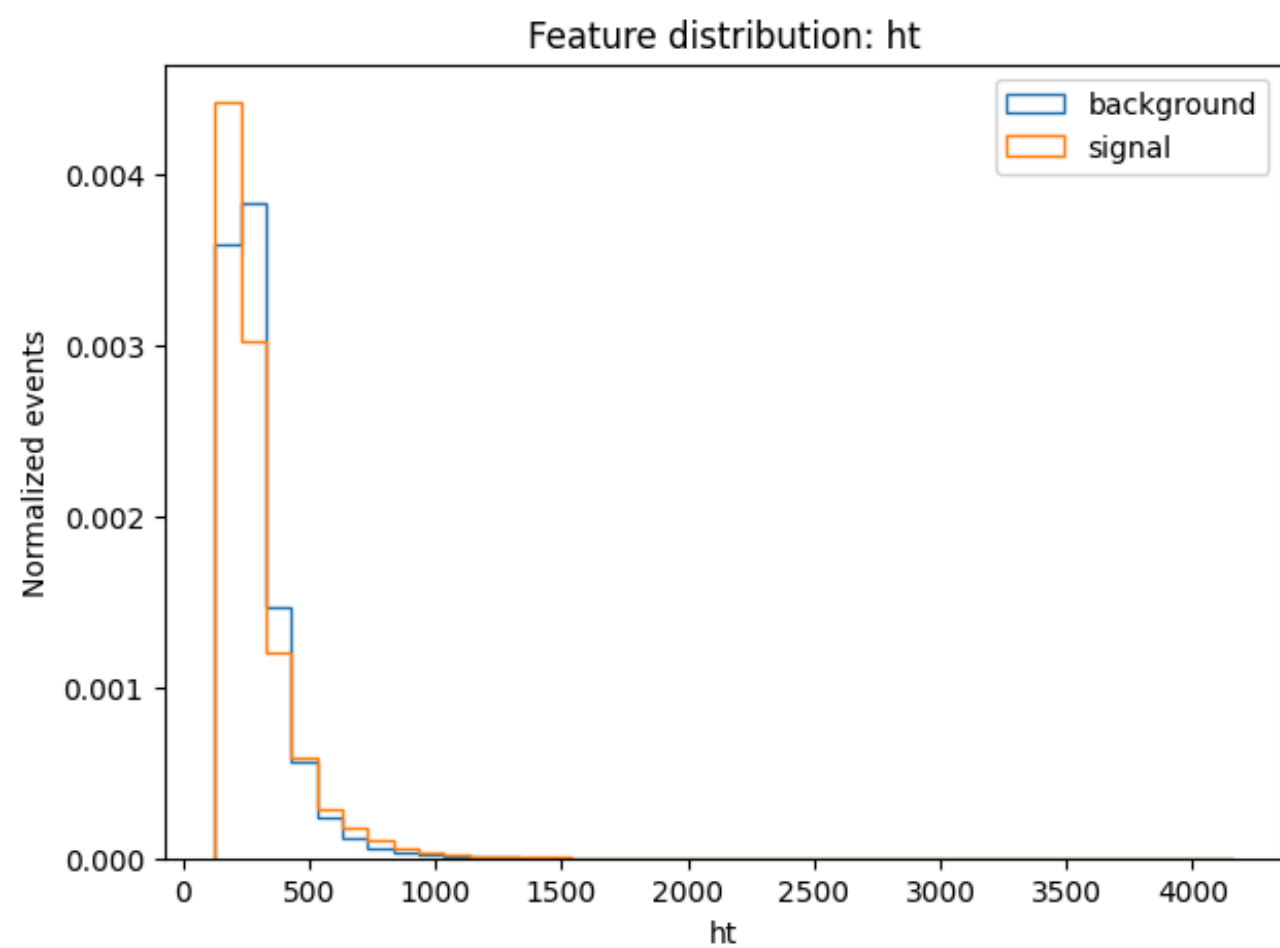
5 rows × 37 columns

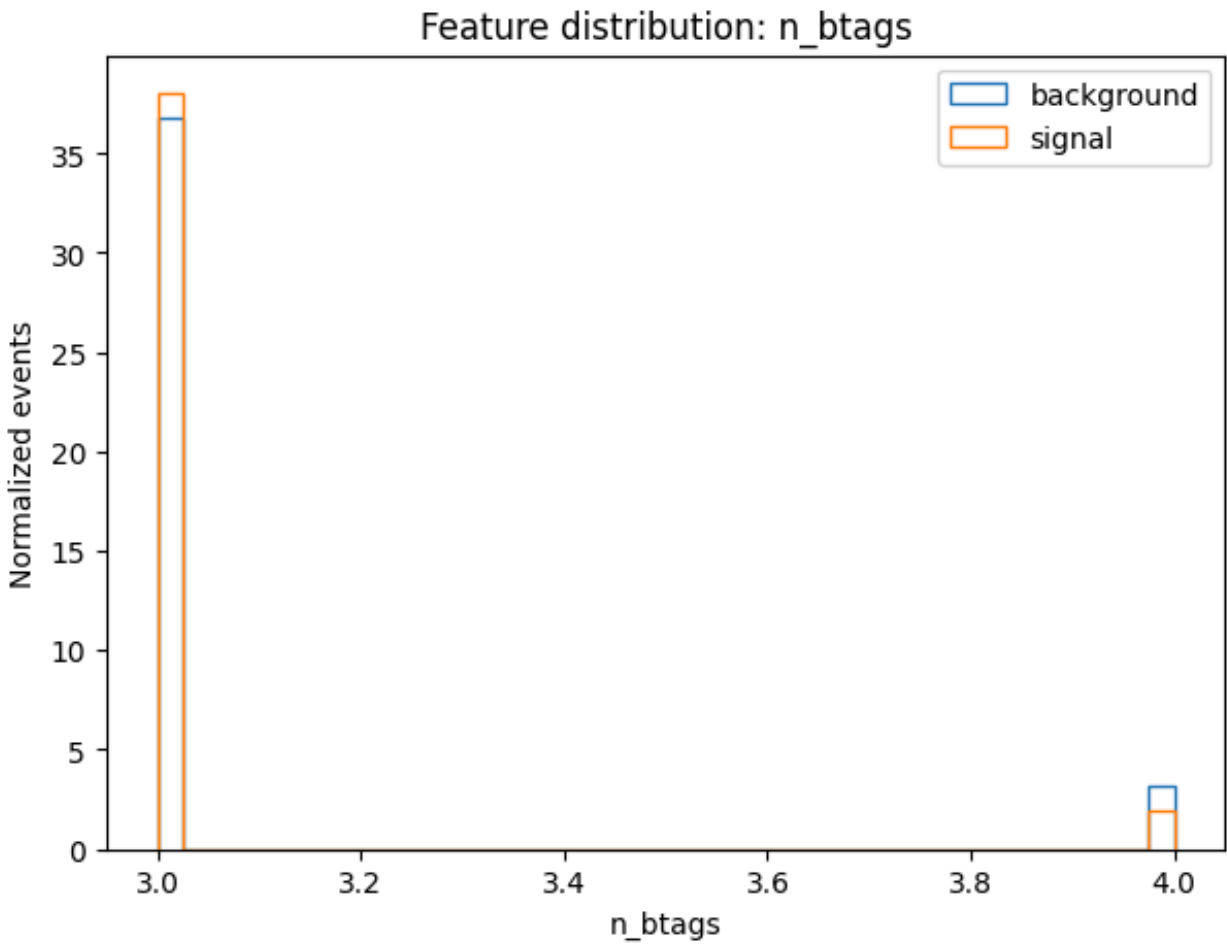
events	
label	
0	64345
1	10781

dtype: int64

```
In [34]: # List of engineered features to plot their distributions
for feature in ["jet1_pt", "jet2_pt", "ht", "m_4j", "m_jj_12", "min_deltaR_jj", "n_btags"]:
    # Skip plotting if the feature does not exist in the DataFrame (e.g., if feature engineering is incomplete)
    if feature not in X.columns:
        continue
    # Get non-NaN values for the current feature
    values = X[feature].dropna().to_numpy()
    # Determine bin edges for the histogram
    bins = np.histogram_bin_edges(values, bins=40)
    plt.figure() # Create a new figure for each plot
    # Plot histogram for background events (y == 0)
    plt.hist(X.loc[y == 0, feature], bins=bins, histtype="step", density=True, label="background")
    # Plot histogram for signal events (y == 1)
    plt.hist(X.loc[y == 1, feature], bins=bins, histtype="step", density=True, label="signal")
    plt.xlabel(feature) # Set x-axis label
    plt.ylabel("Normalized events") # Set y-axis label
    plt.title(f"Feature distribution: {feature}") # Set plot title
    plt.legend() # Display legend
    plt.show() # Show the plot
```







Section 3 - Classification Agent Template Class

`ClassificationAgent` handles shared bookkeeping: feature selection, train/validation/test splitting, optional scaling, metric calculation, and text summaries.

The class handles shared bookkeeping and dispatches to the selected model family:

Method	Input	Output
<code>.train(X, y)</code>	feature dataframe and label series	fitted classification agent with metrics
<code>.predict(X_new)</code>	dataframe with the selected feature columns	signal-like score for each row

Most of the classifier work happens here. Add a new entry to `MODEL_LIBRARY`, then add the matching training block inside `ClassificationAgent.train(...)`. The planning agent can then choose that `model_family`.

```
In [62]: def baseline_strategy(feature_columns):
        """Defines a baseline strategy for the classification agent using a Random Forest model."""
        return {
            "strategy_name": "baseline_random_forest", # Name of the strategy
            "rationale": "Baseline random forest using all engineered four-jet features.", # Explanation for the strategy
            "feature_columns": list(feature_columns), # List of features to use
            "model_family": "random_forest", # Type of model to use
            "hyperparameters": {"n_estimators": 160, "min_samples_leaf": 2, "max_features": "sqrt"}, # Model hyperparameters
        }

class ClassificationAgent:
    """Train and score one classifier chosen by a planning strategy.

    Add models in two places:
    1. Add a new entry to MODEL_LIBRARY.
    2. Add a new `elif self.model_family == "...` block in train().

    predict() already works for scikit-learn classifiers with predict_proba.
    """

    # Class-level dictionary to store configurations for different model families
    MODEL_LIBRARY = {
        "random_forest": {
            "defaults": {"n_estimators": 160, "min_samples_leaf": 2, "max_depth": None, "max_features": "sqrt"}, # Default hyperparameters
            "allowed": {"n_estimators", "min_samples_leaf", "max_depth", "max_features"}, # Allowed hyperparameters for planning agent
            "use_scaler": False, # Whether this model family requires feature scaling
        },
        # YOUR CODE HERE: add another model_family entry, for example "logistic_regression" or "mlp".
        "hist_gradient_boosting": {
            "defaults": {"learning_rate": 0.1, "max_iter": 100, "max_depth": None, "max_leaf_nodes": 31, "min_samples_leaf": 20},
            "allowed": {"learning_rate", "max_iter", "max_depth", "max_leaf_nodes", "min_samples_leaf"},
            "use_scaler": False,
        },
    }

    def __init__(self, strategy=None, random_state=RANDOM_STATE):
        """Initializes the ClassificationAgent with a strategy and random state."""
        self.strategy = strategy # The classification strategy (dictionary)
        self.random_state = random_state # Random seed for reproducibility
        self.name = (strategy or {}).get("strategy_name", "classification_agent") # Agent name, defaults if no strategy
        self.model_family = (strategy or {}).get("model_family", "random_forest") # Model type, defaults to random_forest
        self.feature_names_ = [] # List to store names of features actually used
        self.hyperparameters = {} # Dictionary to store model hyperparameters
        self.scaler = None # StandardScaler instance (if used)
        self.model = None # The trained scikit-learn model
        self.metrics = {} # Dictionary to store evaluation metrics
        self.n_trained = 0 # Counter for how many times the agent has been trained

    @classmethod
    def catalog(cls):
        """Returns a catalog of available model families and their configurations."""
        return {
            name: {
                "defaults": config["defaults"],
                "allowed": config["allowed"],
                "use_scaler": config["use_scaler"],
            }
            for name, config in cls.MODEL_LIBRARY.items()
        }

    def _config(self):
        """Returns the configuration dictionary for the current model_family."""
        if self.model_family not in self.MODEL_LIBRARY:
            raise ValueError(f"Unknown model_family {self.model_family!r}. Choose one of {list(self.MODEL_LIBRARY)}.")
        return self.MODEL_LIBRARY[self.model_family]

    def _clean_hyperparameters(self):
        """Filters and sets hyperparameters based on the strategy and allowed defaults."""
        config = self._config()
        params = config["defaults"].copy() # Start with default hyperparameters
        # Override defaults with values from the strategy if allowed
        for key, value in (self.strategy.get("hyperparameters", {}) or {}).items():
            if key in config["allowed"]:
                params[key] = value
        return params

    def _prepare_data(self, X, y):
        """Prepares data for training: selects features, splits data, and optionally scales it."""
```

```

# If no strategy is provided, use the baseline random forest strategy
if self.strategy is None:
    self.strategy = baseline_strategy(X.columns)
self.name = self.strategy.get("strategy_name", "classification_agent")
self.model_family = self.strategy.get("model_family", "random_forest")

# Select features based on the strategy, ensuring they are present and numeric
requested = self.strategy.get("feature_columns", [])
self.feature_names_ = [col for col in requested if col in X.columns and pd.api.types.is_numeric_dtype(X[col])]
if not self.feature_names_:
    raise ValueError("Classifier strategy did not select any valid numeric feature columns.")
self.hyperparameters = self._clean_hyperparameters() # Clean and set hyperparameters

X_selected = X[self.feature_names_].copy() # Create a DataFrame with only selected features
# Split data into training, validation, and test sets (60/20/20 split)
self.X_train, X_temp, self.y_train, y_temp = train_test_split(
    X_selected, y, test_size=0.40, stratify=y, random_state=self.random_state
)
self.X_val, self.X_test, self.y_val, self.y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=self.random_state
)

# Apply StandardScaler if required by the model family
if self._config()["use_scaler"]:
    self.scaler = StandardScaler()
    self.X_train_model = self.scaler.fit_transform(self.X_train) # Fit and transform training data
else:
    self.scaler = None
    self.X_train_model = self.X_train # Use original training data if no scaling

def _model_input(self, X):
    """Prepares input `X` for prediction, applying feature selection and scaling."""
    X_selected = X[self.feature_names_].copy() # Select features
    # Apply scaler if it exists, otherwise return selected features
    return self.scaler.transform(X_selected) if self.scaler is not None else X_selected

def train(self, X, y):
    """Trains the classifier model based on the selected strategy."""
    self._prepare_data(X, y) # Prepare data before training

    if self.model_family == "random_forest":
        params = self.hyperparameters.copy()
        # Validate and clip hyperparameters for Random Forest
        params["n_estimators"] = int(np.clip(int(params.get("n_estimators", 160)), 50, 300))
        params["min_samples_leaf"] = int(np.clip(int(params.get("min_samples_leaf", 2)), 1, 20))
        if params.get("max_depth") is not None:
            params["max_depth"] = int(np.clip(int(params["max_depth"]), 2, 40))
        if params.get("max_features") not in ["sqrt", "log2", None]:
            params["max_features"] = "sqrt"
        # Initialize and fit RandomForestClassifier
        self.model = RandomForestClassifier(random_state=self.random_state, n_jobs=-1, **params)
        self.model.fit(self.X_train_model, self.y_train)

# YOUR CODE HERE: add `elif self.model_family == "your_model_name":` blocks.
# Each block should set `params`, create `self.model`, and fit it on self.X_train_model.
elif self.model_family == "hist_gradient_boosting":
    params = self.hyperparameters.copy()
    params["max_iter"] = int(np.clip(int(params.get("max_iter", 300)), 50, 1000))
    params["learning_rate"] = float(np.clip(float(params.get("learning_rate", 0.1)), 0.01, 1.0))
    params["min_samples_leaf"] = int(np.clip(int(params.get("min_samples_leaf", 20)), 1, 100))
    if params.get("max_depth") is not None:
        params["max_depth"] = int(np.clip(int(params["max_depth"]), 2, 40))
    self.model = HistGradientBoostingClassifier(random_state=self.random_state, **params)
    self.model.fit(self.X_train_model, self.y_train)
else:
    raise ValueError(f"No training code is defined for model_family {self.model_family!r}.")

self.hyperparameters = params # Update hyperparameters with final (clipped) values
self.n_trained += 1 # Increment training counter
# Evaluate model on validation and test sets and store metrics
self.metrics["validation"] = self.evaluate(self.X_val, self.y_val)
self.metrics["test"] = self.evaluate(self.X_test, self.y_test)
return self

def predict(self, X):
    """Generates class probabilities (signal-like scores) for new data."""
    # Get raw prediction probabilities, taking the probability of the positive class (1)
    scores = self.model.predict_proba(self._model_input(X))[:, 1]
    # Clip scores to be between 0.0 and 1.0 to ensure valid probabilities
    return np.clip(np.asarray(scores, dtype=float), 0.0, 1.0)

def evaluate(self, X, y):
    """Evaluates the model's performance on given data (X, y)."""
    scores = self.predict(X) # Get prediction scores
    # Convert scores to binary predictions using a 0.5 threshold
    predictions = (scores >= 0.5).astype(int)
    return {
        "accuracy": accuracy_score(y, predictions), # Overall accuracy
        "auc": roc_auc_score(y, scores), # Area Under ROC Curve
        "confusion_matrix": confusion_matrix(y, predictions), # Confusion matrix
        "classification_report": classification_report(y, predictions, zero_division=0), # Detailed classification report
        "scores": scores, # Raw prediction scores
    }

def describe(self):
    """Returns a string describing the ClassificationAgent's current state and performance."""
    val_auc = self.metrics.get("validation", {}).get("auc", np.nan)
    test_auc = self.metrics.get("test", {}).get("auc", np.nan)
    return (
        f"ClassificationAgent `{self.name}` trained `{self.model_family}` using {len(self.feature_names_)} features. "
        f"Validation AUC = {val_auc:.3f}; test AUC = {test_auc:.3f}. "
        f"Hyperparameters: {self.hyperparameters}."
    )

# Get the catalog of available models from the ClassificationAgent class
AGENT_CATALOG = ClassificationAgent.catalog()

# Create a baseline classifier instance using the defined baseline strategy
baseline_classifier = ClassificationAgent(strategy=baseline_strategy(X.columns))
# Train the baseline classifier using the full feature set X and labels y
baseline_classifier.train(X, y)
# Get prediction scores for the test set from the baseline classifier
baseline_scores = baseline_classifier.predict(baseline_classifier.X_test)
# Print a description of the baseline classifier's training and performance
print(baseline_classifier.describe())

```

ClassificationAgent `baseline\_random\_forest` trained `random\_forest` using 37 features. Validation AUC = 0.811; test AUC = 0.811. Hyperparameters: {'n_estimators': 160, 'min_samples_leaf': 2, 'max_depth': None, 'max_features': 'sqrt'}.

```

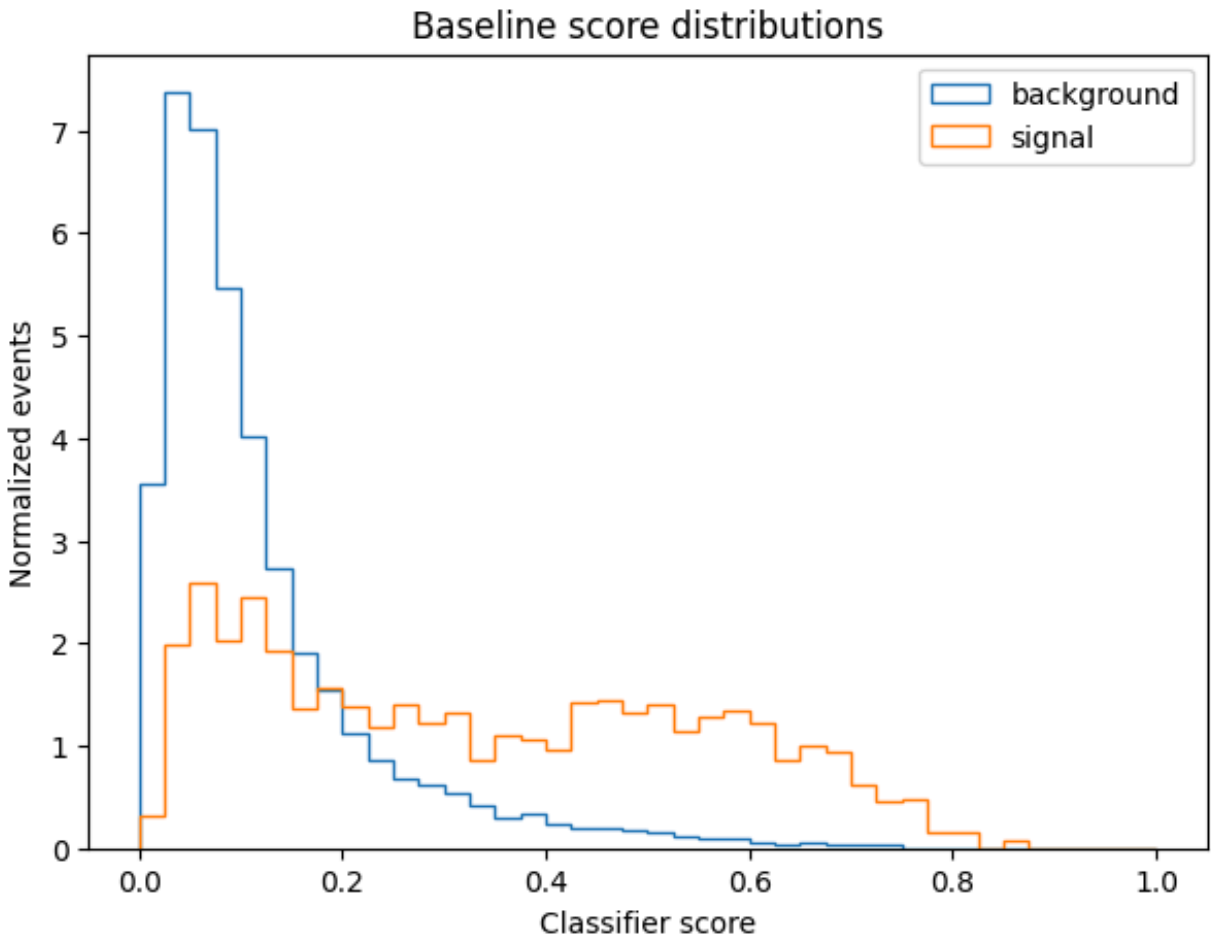
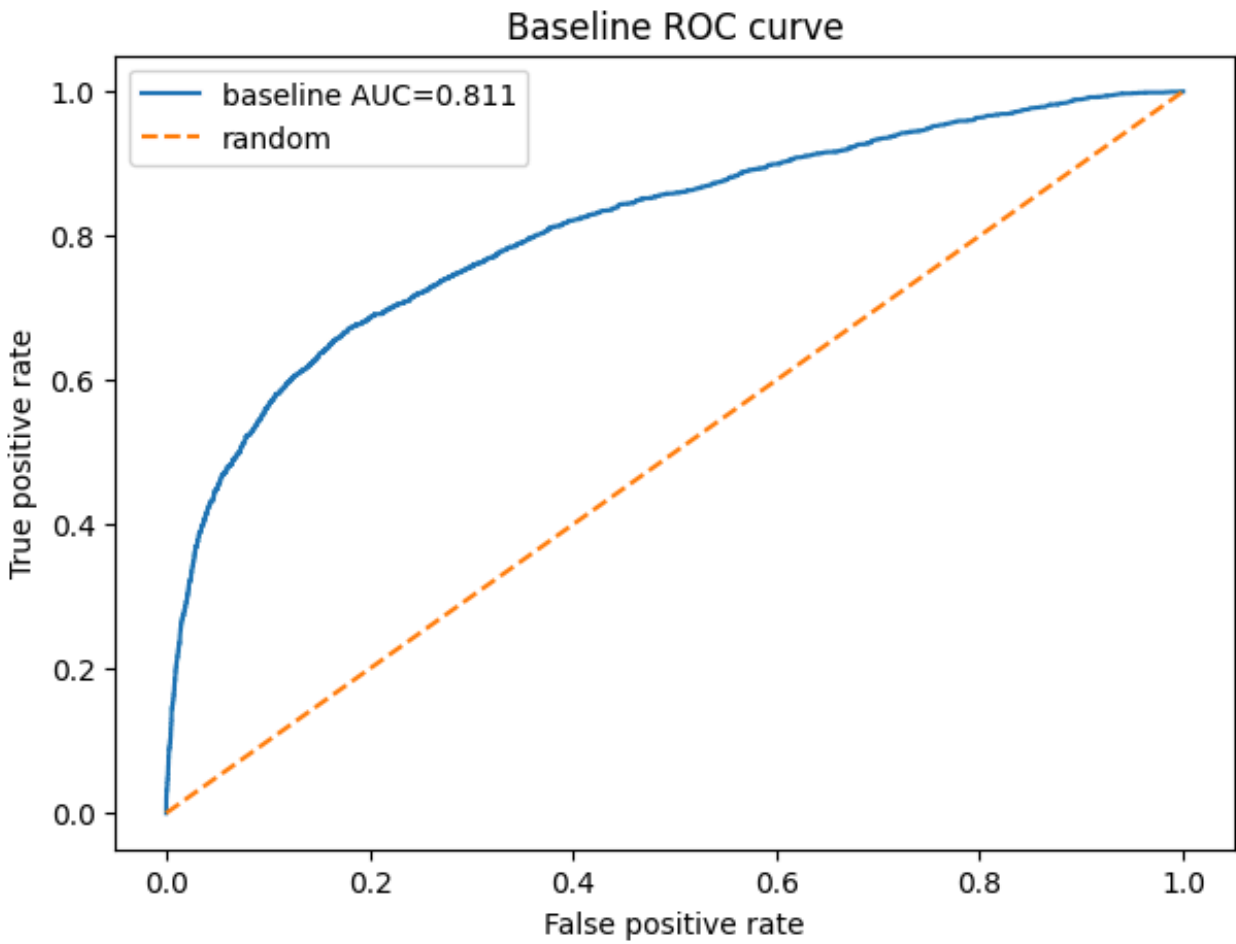
In [63]: # Calculate False Positive Rate (fpr), True Positive Rate (tpr), and thresholds for the ROC curve
fpr, tpr, _ = roc_curve(baseline_classifier.y_test, baseline_scores)

plt.figure() # Create a new figure
# Plot the ROC curve
plt.plot(fpr, tpr, label=f"baseline AUC={baseline_classifier.metrics['test']['auc']:.3f}")

```

```
# Plot the diagonal line representing a random classifier
plt.plot([0, 1], [0, 1], linestyle="--", label="random")
plt.xlabel("False positive rate") # Set x-axis label
plt.ylabel("True positive rate") # Set y-axis label
plt.title("Baseline ROC curve") # Set plot title
plt.legend() # Display legend
plt.show() # Show the plot

# Define bins for classifier score distributions from 0 to 1
score_bins = np.linspace(0, 1, 41)
plt.figure() # Create a new figure
# Plot histogram of scores for background events (y_test == 0)
plt.hist(baseline_scores[baseline_classifier.y_test.to_numpy() == 0], bins=score_bins, histtype="step", density=True, label="background")
# Plot histogram of scores for signal events (y_test == 1)
plt.hist(baseline_scores[baseline_classifier.y_test.to_numpy() == 1], bins=score_bins, histtype="step", density=True, label="signal")
plt.xlabel("Classifier score") # Set x-axis label
plt.ylabel("Normalized events") # Set y-axis label
plt.title("Baseline score distributions") # Set plot title
plt.legend() # Display legend
plt.show() # Show the plot
```



Section 4 - Signal Analysis Agent Template Class

`SignalAnalysisAgent` turns classifier scores into a simple counting analysis.

Fill in the threshold scan yourself. For each threshold, select events with `scores >= threshold`, count selected signal and background events, compute efficiencies, and compute the counting significance $S/\sqrt{B}$.

```
In [64]: class SignalAnalysisAgent:
    """Performs a threshold scan on classifier scores to find the optimal significance (S/sqrt(B))."""
    def __init__(self, thresholds=None):
        # Initialize thresholds; default to steps of 0.1 from 0.1 to 0.9
        self.thresholds = thresholds if thresholds is not None else np.arange(0.1, 1.0, 0.1)
        self.scan_table = None # DataFrame to store results of the threshold scan
        self.best = None # Dictionary to store the best threshold and its metrics

    def analyze(self, y_true, scores):
        """Analyzes the signal significance (S/sqrt(B)) across different classification thresholds.
        """
        # Convert y_true to a Series and reset index for consistent operations
        y_true = pd.Series(y_true).astype(int).reset_index(drop=True)
        scores = np.asarray(scores, dtype=float) # Convert scores to numpy array
        # Calculate total number of signal and background events, ensuring at least 1 to avoid division by zero
        total_signal = max(int((y_true == 1).sum()), 1)
        total_background = max(int((y_true == 0).sum()), 1)
        rows = [] # List to collect results for each threshold

        # YOUR CODE HERE: loop over self.thresholds.
        # For each threshold:
        # - select events with scores >= threshold
        # - count selected signal events S
        # - count selected background events B
        # - compute signal efficiency S / total_signal
        # - compute background efficiency B / total_background
        # - compute counting significance S / sqrt(B)
        # - append one dictionary to rows with the columns used below
        for threshold in self.thresholds:
            # Select events where classifier score is greater than or equal to the current threshold
            selected_mask = (scores >= threshold)
            selected_y = y_true[selected_mask]

            S = selected_y.sum() # Count selected signal events (where y_true is 1)
            B = len(selected_y) - S # Count selected background events (total selected - signal selected)

            sig_eff = S / total_signal # Signal efficiency
            back_eff = B / total_background # Background efficiency

            # Calculate counting significance S / sqrt(B)
```

```
# Handle cases where B might be zero to avoid RuntimeWarning or division by zero
count_sig = S / np.sqrt(B) if B > 0 else np.nan

rows.append({"threshold": threshold, "S": S, "B": B,
            "signal_efficiency": sig_eff, "background_efficiency": back_eff,
            "S_over_sqrt_B": count_sig})

self.scan_table = pd.DataFrame(rows) # Convert list of dictionaries to DataFrame
# Raise an error if the scan table is empty (e.g., if the TODOs were not completed)
if self.scan_table.empty:
    raise ValueError("Threshold scan is empty. Finish SignalAnalysisAgent.analyze(...).")
# Find the row with the maximum S_over_sqrt_B, handling NaN values correctly
self.best = self.scan_table.sort_values("S_over_sqrt_B", ascending=False, na_position="last").iloc[0]
return self.scan_table, self.best

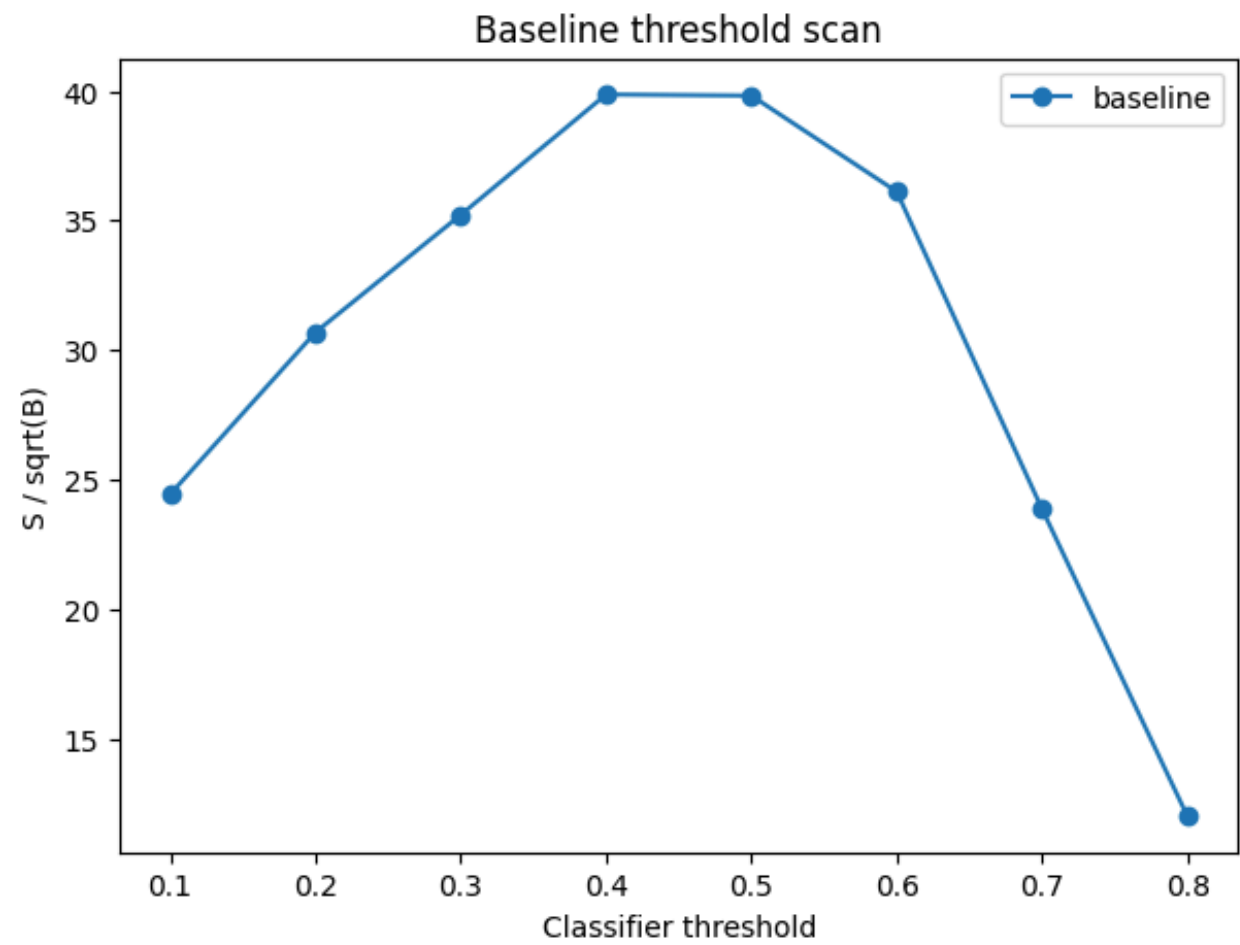
def describe(self):
    """Returns a string describing the best result from the signal analysis."""
    return (
        f"SignalAnalysisAgent found the best threshold at score >= {self.best['threshold']:.2f}, "
        f"with S/sqrt(B) = {self.best['S_over_sqrt_B']:.2f}."
    )

# Create an instance of SignalAnalysisAgent
signal_agent = SignalAnalysisAgent()
# Analyze the baseline classifier's test scores
baseline_scan, baseline_best = signal_agent.analyze(baseline_classifier.y_test, baseline_scores)
# Display the full threshold scan table
display(baseline_scan)
# Print a description of the best result from the baseline analysis
print(signal_agent.describe())

plt.figure() # Create a new figure
# Plot the S/sqrt(B) versus classifier threshold
plt.plot(baseline_scan["threshold"], baseline_scan["S_over_sqrt_B"], marker="o", label="baseline")
plt.xlabel("Classifier threshold") # Set x-axis label
plt.ylabel("S / sqrt(B)") # Set y-axis label
plt.title("Baseline threshold scan") # Set plot title
plt.legend() # Display legend
plt.show() # Show the plot
```

	threshold	S	B	signal_efficiency	background_efficiency	S_over_sqrt_B
0	0.1	1784	5334	0.827458	0.414452	24.426899
1	0.2	1391	2055	0.645176	0.159674	30.684653
2	0.3	1111	995	0.515306	0.077312	35.221068
3	0.4	878	485	0.407236	0.037685	39.867926
4	0.5	600	227	0.278293	0.017638	39.823399
5	0.6	321	79	0.148887	0.006138	36.115322
6	0.7	104	19	0.048237	0.001476	23.859236
7	0.8	12	1	0.005566	0.000078	12.000000
8	0.9	0	0	0.000000	0.000000	NaN

SignalAnalysisAgent found the best threshold at score >= 0.40, with S/sqrt(B) = 39.87.



Section 5 - Planning Agent Template Class

The planning agent reads previous round results and proposes the next classifier strategy. It does not write executable classifier code.

The API call, JSON parsing, and validation are fixed infrastructure. Edit `PLANNING_PROMPT` to direct the strategy: which features to try, which model families to prefer, how much to explore, and how to react to previous results.

Example planning-agent output:

```
{
  "strategy_name": "extra_trees_mass_and_btag_scan",
  "rationale": "Try a high-capacity tree model using invariant-mass, angular-separation, and b-tag variables that match the four-b signal topology.",
  "feature_columns": [
    "ht",
    "m_4j",
    "m_jj_12",
    "m_jj_13",
    "m_jj_14",
    "m_jj_23",
    "m_jj_24",
    "m_jj_34",
    "min_deltaR_jj",
    "mean_deltaR_jj",
    "n_btags",
  ],
  "model_family": "extra_trees",
  "hyperparameters": {
    "n_estimators": 220,
    "min_samples_leaf": 1,
    "max_features": "sqrt"
  }
}
```

```
In [65]: PLANNING_PROMPT = """
Return one valid JSON strategy for the next classifier attempt.
The JSON must contain exactly these keys: strategy_name, rationale, feature_columns, model_family, hyperparameters.
Use only feature_columns from the provided feature list.
Use only a model_family from the provided model catalog.

# YOUR CODE HERE: add strategy instructions for the planning agent.
# Examples of strategy guidance:
# - Which physics features should it try first?
# - Should it explore new model families or tune the current best one?
# - Should it prefer simple models, high-capacity models, or MLPs?
# - How should it use the previous round history?

- Start with physics-motivated features: n_btags, invariant masses (m_jj_*), angular separations (deltaR_jj_*), and ht. These capture the H->aa->4b topology.
- Try random_forest first, then switch to hist_gradient_boosting in later rounds - it often performs better on tabular data.
- Avoid raw jet pt/eta/phi - the derived features (masses, angles, b-tags) carry more signal-vs-background information.
- If a previous round's approach gave poor AUC, try a different model family rather than just tuning hyperparameters. If AUC was good, try tuning around those hyperparameters.
- Keep feature sets focused (8-15 features) rather than using all 37.

""".strip()

class PlanningAgent:
    """Agent responsible for generating classification strategies using an LLM."""
    def __init__(self, prompt=PLANNING_PROMPT, model_templates=AGENT_CATALOG, api_key=CEREBRAS_API_KEY, model_name=CEREBRAS_MODEL):
        self.prompt = prompt # The prompt used to guide the LLM's strategy generation
        self.model_templates = model_templates # Catalog of available classification models
        self.api_key = api_key # API key for the Cerebras LLM service
        self.model_name = model_name # Name of the Cerebras LLM model to use
        self.last_plan = None # Stores the last generated plan

    def plan(self, history, dataset_summary, baseline_summary):
        """Requests a new strategy from the Cerebras LLM based on history and summaries."""
        self.last_plan = request_strategy_from_cerebras(
            prompt=self.prompt, # The main prompt for the LLM
            history=history, # List of past round summaries (for contextual planning)
            dataset_summary=dataset_summary, # Summary of the current dataset
            baseline_summary=baseline_summary, # Summary of the baseline model's performance
            model_templates=self.model_templates, # Catalog of models for the LLM to choose from
            api_key=self.api_key, # API key
            model_name=self.model_name, # Model name
        )
        return self.last_plan

    def describe(self, plan=None):
        """Returns a string description of a given plan (or the last plan)."""
        plan = plan or self.last_plan
        if not plan: # Handle case where no plan has been generated yet
            return "No plan available."
        return (
            f"PlanningAgent proposed `{plan['strategy_name']}`: a `{plan['model_family']}` strategy "
            f"using {len(plan['feature_columns'])} features. Rationale: {plan['rationale']}"
        )

    def summarize_round(self, round_id, plan, classifier_agent, best):
        """Summarizes the results of a planning round for use in history."""
        return summarize_planning_round(round_id, plan, classifier_agent, best)
```

Baseline Agentic Workflow

This cell runs the same agent sequence once without an LLM-planned strategy. It gives a baseline random-forest result to compare against later.

Inputs: `X`, `y`, and `baseline_classifier` from the previous sections.

Outputs: `baseline_summary`, `baseline_scan`, `baseline_best`, and diagnostic plots.

```
In [66]: # Create a dictionary summarizing the baseline model's performance metrics
baseline_summary = {
    "model": baseline_classifier.name, # Name of the baseline model
    "test_auc": baseline_classifier.metrics["test"]["auc"], # AUC on the test set
    "best_threshold": baseline_best["threshold"], # Best threshold from the signal analysis
    "best_S_over_sqrt_B": baseline_best["S_over_sqrt_B"], # Best S/sqrt(B) value
}
# Display the baseline summary as a pandas DataFrame for better readability
display(pd.DataFrame([baseline_summary]))
```

	model	test_auc	best_threshold	best_S_over_sqrt_B
0	baseline_random_forest	0.811072	0.4	39.867926

Agentic Planning Loop

Complete the orchestration loop. The loop should pass information from one agent to the next, save each round result, and update `history` so the planning agent can react to previous attempts.

```
In [67]: # Define the number of planning rounds to execute
NUM_PLANNING_ROUNDS = 4
history = [] # List to store summaries of previous planning rounds
planned_results = [] # List to store detailed results of each planned strategy
planning_agent = PlanningAgent() # Initialize the PlanningAgent

# Loop for the specified number of planning rounds
for round_id in range(1, NUM_PLANNING_ROUNDS + 1):
    print(f"\nPlanning round {round_id}")

    # YOUR CODE HERE: create a FeatureAgent and build X, y from selected_events.
    # Create a new FeatureAgent instance for each round
    feature_agent = FeatureAgent()
    # Generate features (X) and labels (y) from the raw selected_events
    X, y = feature_agent.features(selected_events)
    print(feature_agent.describe()) # Print description of the features created

    # YOUR CODE HERE: build dataset_summary for the planning agent.
    # It should include event count, feature count, class counts, and feature names.
    dataset_summary = {
        "n_events": len(X), # Total number of events
        "n_features": X.shape[1], # Number of features
        "n_signal": int(y.sum()), # Count of signal events (label == 1)
        "n_background": int((1 - y).sum()), # Count of background events (label == 0)
        "feature_names": list(X.columns), # List of all feature names
    }

    # YOUR CODE HERE: ask the planning agent for the next strategy.
    # Call the planning agent to get a new strategy based on history and summaries
    plan = planning_agent.plan(history=history, dataset_summary=dataset_summary, baseline_summary=baseline_summary)
    print(planning_agent.describe(plan)) # Print a description of the proposed plan

    # YOUR CODE HERE: train the ClassificationAgent using the planned strategy.
    # Create a ClassificationAgent with the newly planned strategy
    classification_agent = ClassificationAgent(strategy=plan)
    # Train the classification agent using the generated features and labels
    classification_agent.train(X, y)
```

```
# Get prediction scores on the test set from the trained classifier
scores = classification_agent.predict(classification_agent.X_test)
print(classification_agent.describe()) # Print a description of the classification agent's performance

# YOUR CODE HERE: run the SignalAnalysisAgent threshold scan.
# Create a SignalAnalysisAgent instance
signal_agent = SignalAnalysisAgent()
# Analyze the classifier's scores to find the best signal sensitivity
scan, best = signal_agent.analyze(classification_agent.y_test, scores)
print(signal_agent.describe()) # Print a description of the best signal analysis result

# YOUR CODE HERE: save the full round result in planned_results.
# Store all relevant information from the current round
planned_results.append({
    "plan": plan, # The strategy plan itself
    "agent": classification_agent, # The trained classification agent
    "scan": scan, # The full threshold scan table
    "scores": scores, # The raw prediction scores
    "best": best # The best result from the signal analysis
})

# YOUR CODE HERE: summarize the round and append it to history.
# Create a concise summary of the current round for the planning agent's history
round_summary = planning_agent.summarize_round(round_id, plan, classification_agent, best)
history.append(round_summary) # Add the summary to the history list

# Check to ensure the planned_results list is populated (sanity check for TODOs)
if len(planned_results) < round_id:
    raise NotImplementedError("Complete the orchestration TODOs above before plotting round results.")

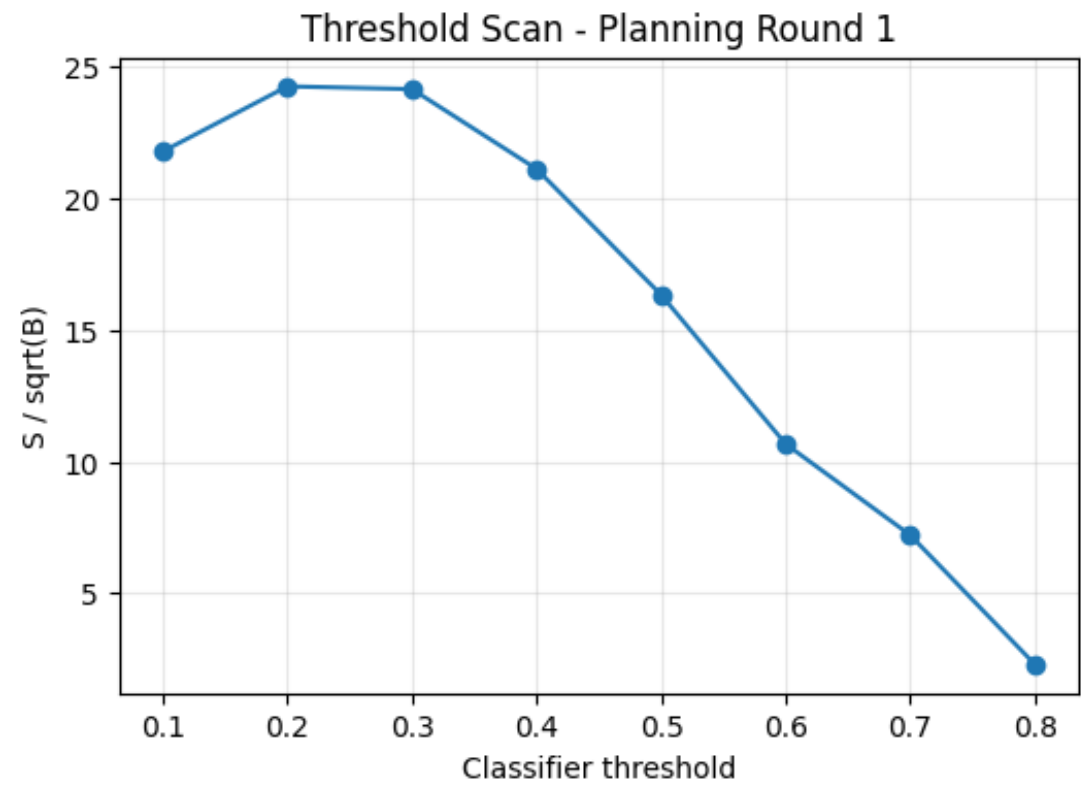
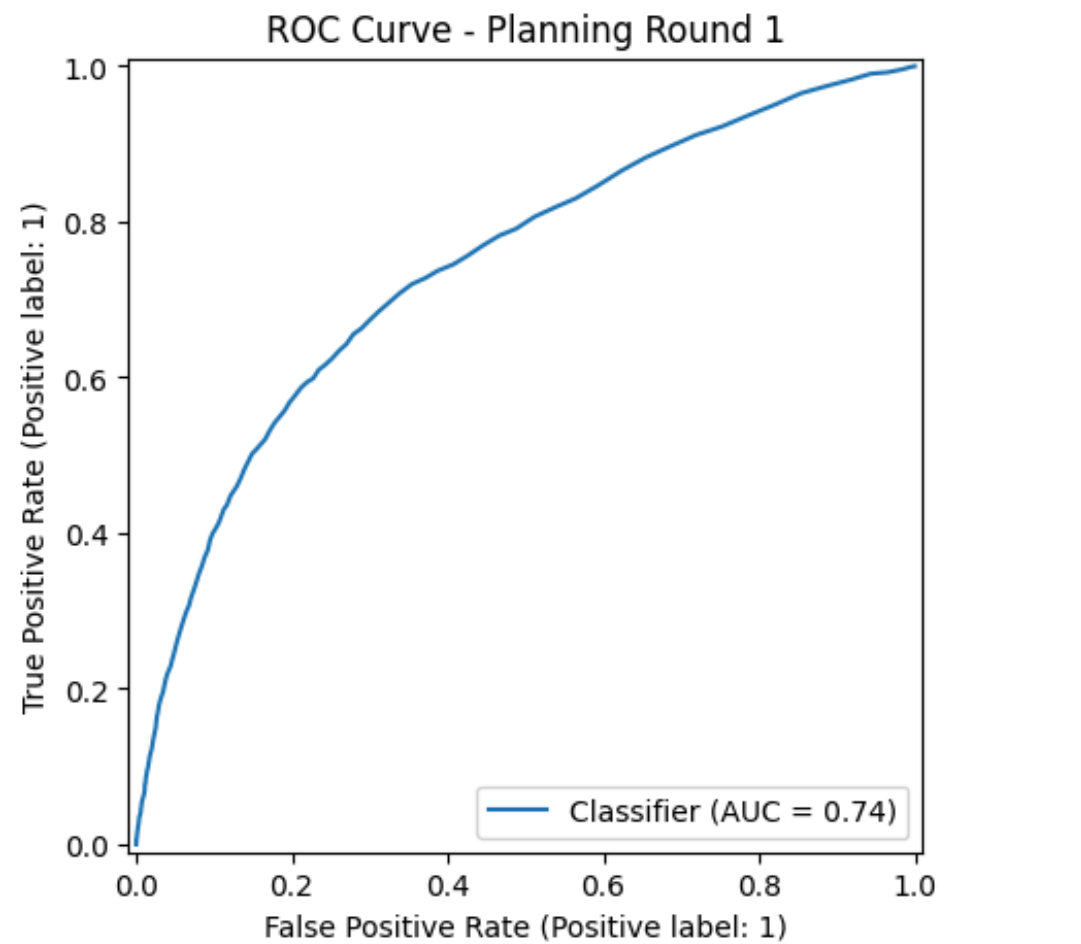
# The displays below should work after the variables above are defined.
# Display the history of planning rounds
display(pd.DataFrame(history))

# Plot the ROC curve for the current round
RocCurveDisplay.from_predictions(classification_agent.y_test, scores)
plt.title(f"ROC Curve - Planning Round {round_id}")
plt.show()

# Plot the threshold scan (S/sqrt(B) vs. threshold) for the current round
plt.figure(figsize=(6, 4))
plt.plot(scan["threshold"], scan["S_over_sqrt_B"], marker="o")
plt.xlabel("Classifier threshold")
plt.ylabel("S / sqrt(B)")
plt.title(f"Threshold Scan - Planning Round {round_id}")
plt.grid(True, alpha=0.3)
plt.show()
```

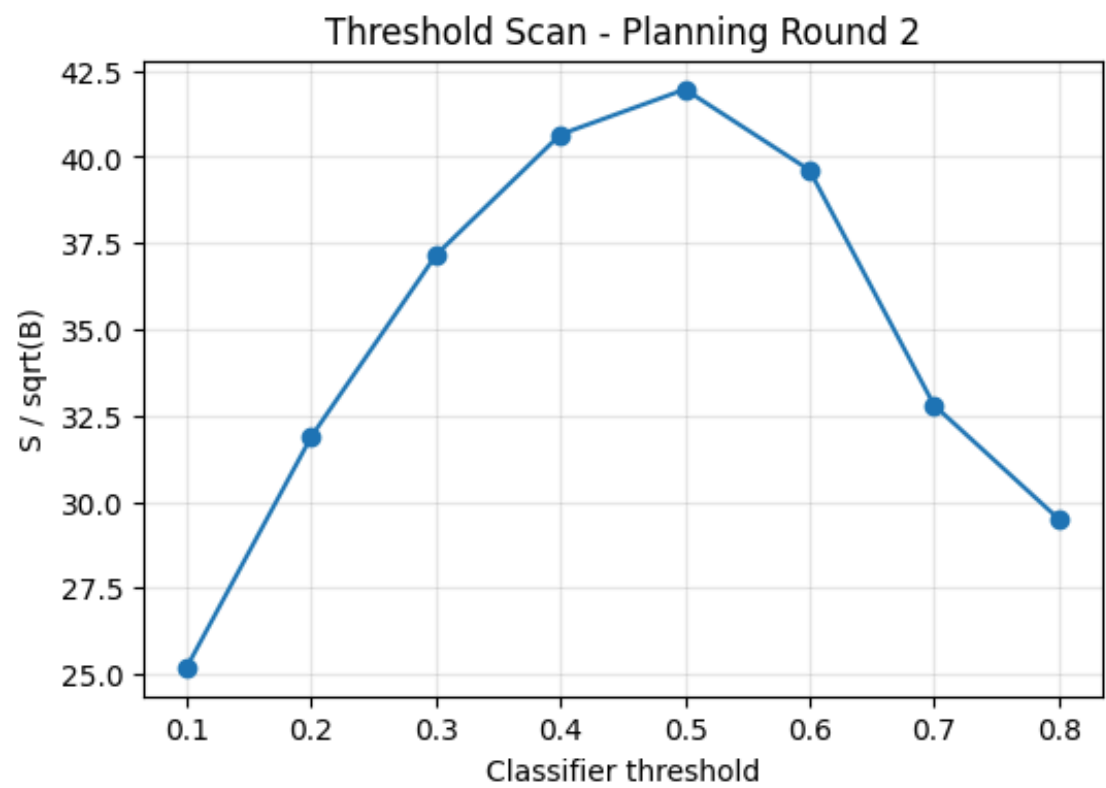
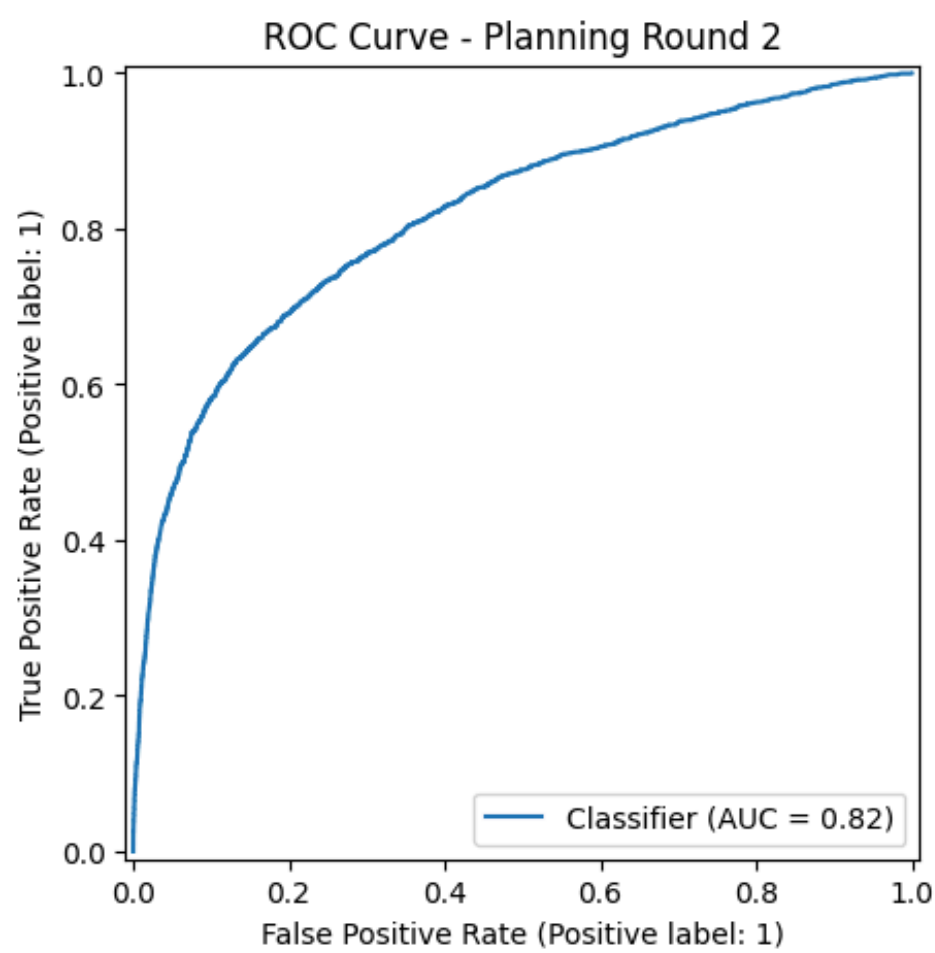
Planning round 1
FeatureAgent `four\_jet\_feature\_agent` built 37 ML features for 75,126 selected events.
PlanningAgent proposed `physics\_mass\_random\_forest`: a `random\_forest` strategy using 13 features. Rationale: Use invariant masses, angular separations, and b-tag multiplicity because they reflect the four-b signal topology.
ClassificationAgent `physics\_mass\_random\_forest` trained `random\_forest` using 13 features. Validation AUC = 0.745; test AUC = 0.742. Hyperparameters: {'n_estimators': 200, 'min_samples_leaf': 1, 'max_depth': None, 'max_features': 'sqrt'}.
SignalAnalysisAgent found the best threshold at score >= 0.20, with S/sqrt(B) = 24.23.

	round	strategy	model_family	features_used	hyperparameters	test_auc	best_threshold	best_S_over_sqrt_B	rationale
0	1	physics_mass_random_forest	random_forest	13	{ "max_depth": null, "max_features": "sqrt", "m...	0.741719	0.2	24.234381	Use invariant masses, angular separations, and...



Planning round 2
FeatureAgent `four\_jet\_feature\_agent` built 37 ML features for 75,126 selected events.
PlanningAgent proposed `exotic\_higgs\_classifier`: a `random\_forest` strategy using 20 features. Rationale: Try to classify exotic-Higgs signal events from QCD heavy-flavour background events using invariant masses, angular separations, and b-tag multiplicity.
ClassificationAgent `exotic\_higgs\_classifier` trained `random\_forest` using 20 features. Validation AUC = 0.814; test AUC = 0.818. Hyperparameters: {'n_estimators': 180, 'min_samples_leaf': 2, 'max_depth': None, 'max_features': 'sqrt'}.
SignalAnalysisAgent found the best threshold at score >= 0.50, with S/sqrt(B) = 41.96.

round		strategy	model_family	features_used	hyperparameters	test_auc	best_threshold	best_S_over_sqrt_B	rationale
0	1	physics_mass_random_forest	random_forest	13	{"max_depth": null, "max_features": "sqrt", "m...	0.741719	0.2	24.234381	Use invariant masses, angular separations, and...
1	2	exotic_higgs_classifier	random_forest	20	{"max_depth": null, "max_features": "sqrt", "m...	0.817933	0.5	41.955002	Try to classify exotic-Higgs signal events fro...



Planning round 3

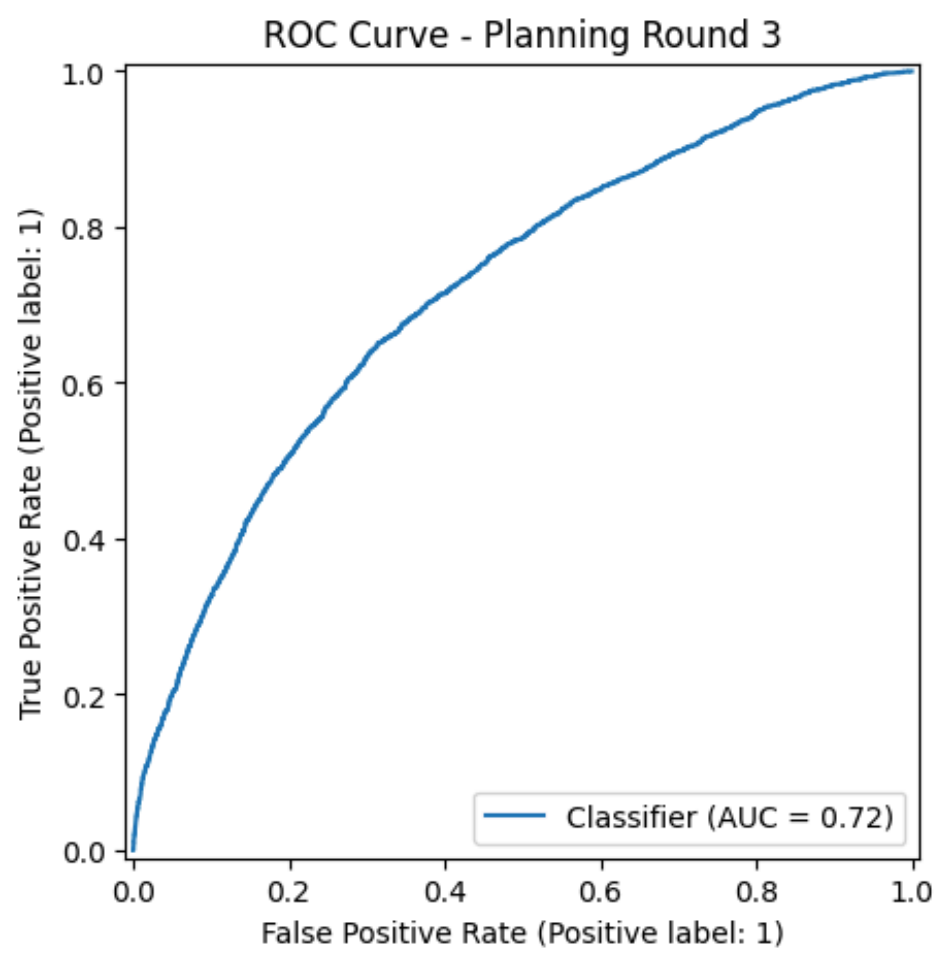
FeatureAgent `four\_jet\_feature\_agent` built 37 ML features for 75,126 selected events.

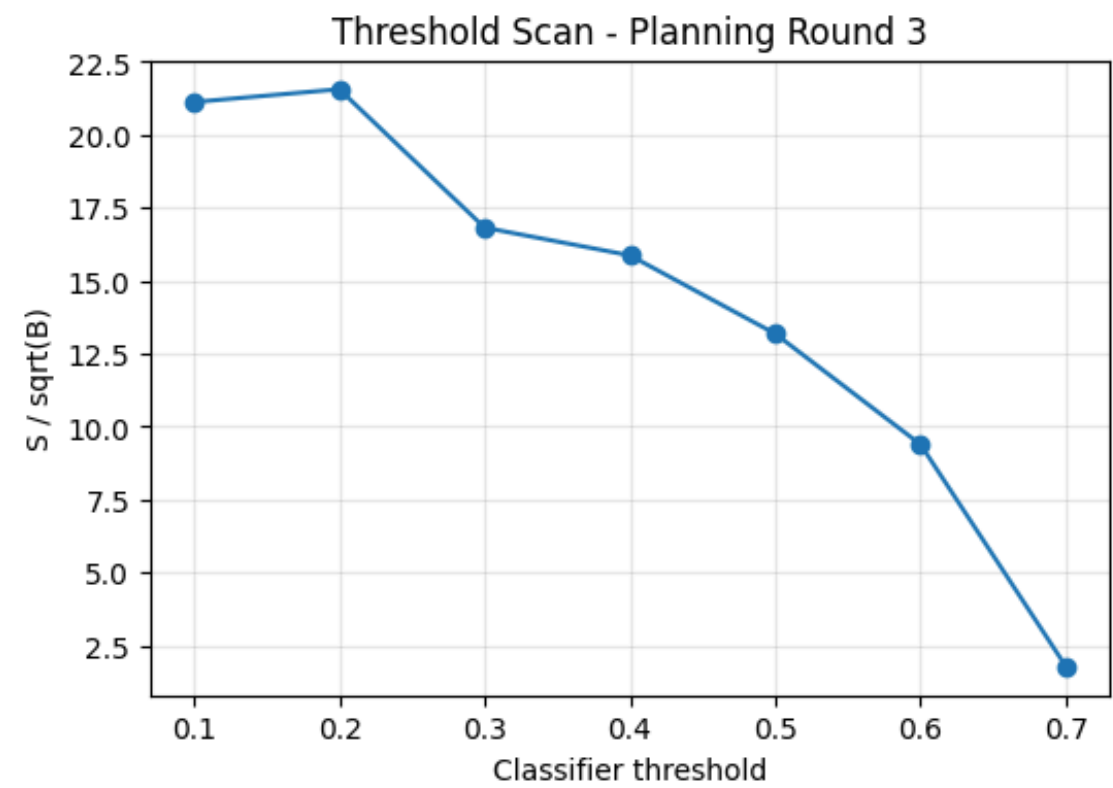
PlanningAgent proposed `physics\_mass\_hist\_gradient\_boosting`: a `hist\_gradient\_boosting` strategy using 6 features. Rationale: Try hist_gradient_boosting with invariant masses, angular separations, and b-tag multiplicity to classify exotic-Higgs signal events from QCD heavy-flavour background events because hist_gradient_boosting often performs better on tabular data.

ClassificationAgent `physics\_mass\_hist\_gradient\_boosting` trained `hist\_gradient\_boosting` using 6 features. Validation AUC = 0.723; test AUC = 0.719. Hyperparameters: {'learning_rate': 0.1, 'max_iter': 100, 'max_depth': None, 'max_leaf_nodes': 31, 'min_samples_leaf': 20}.

SignalAnalysisAgent found the best threshold at score ≥ 0.20 , with $S/\sqrt{B}$ = 21.56.

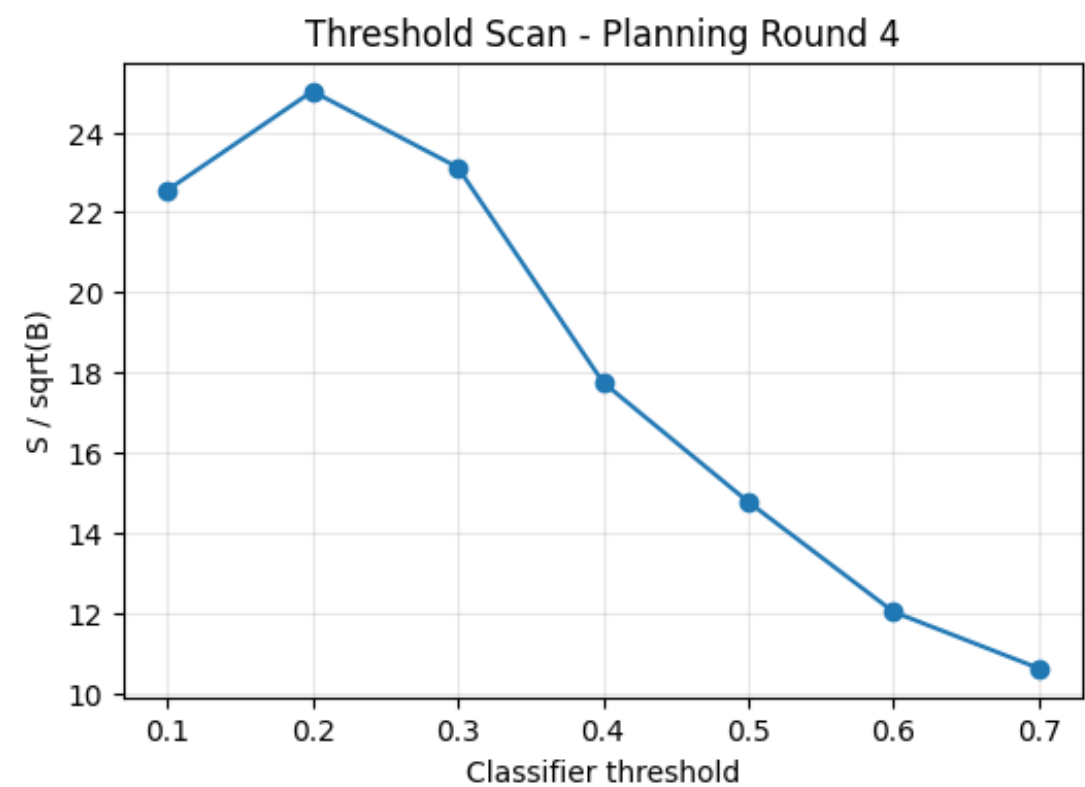
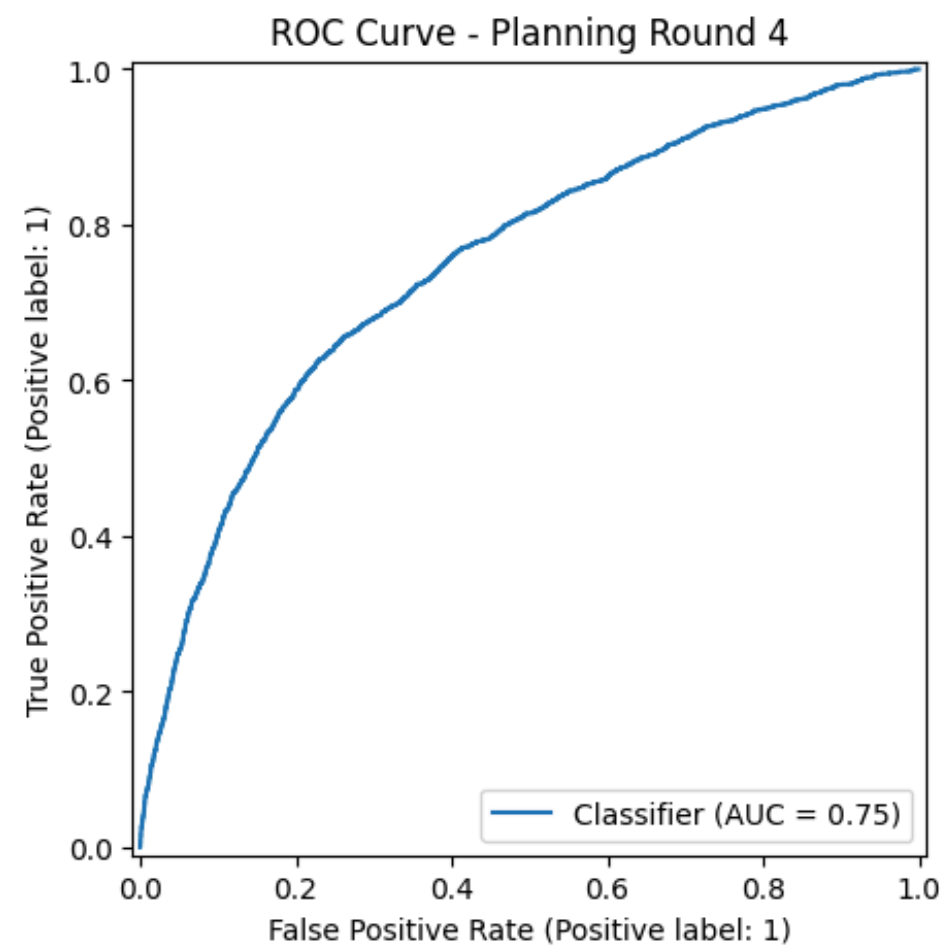
round		strategy	model_family	features_used	hyperparameters	test_auc	best_threshold	best_S_over_sqrt_B	rationale
0	1	physics_mass_random_forest	random_forest	13	{"max_depth": null, "max_features": "sqrt", "m...	0.741719	0.2	24.234381	Use invariant masses, angular separations, and...
1	2	exotic_higgs_classifier	random_forest	20	{"max_depth": null, "max_features": "sqrt", "m...	0.817933	0.5	41.955002	Try to classify exotic-Higgs signal events fro...
2	3	physics_mass_hist_gradient_boosting	hist_gradient_boosting	6	{"learning_rate": 0.1, "max_depth": null, "max...	0.718856	0.2	21.563611	Try hist_gradient_boosting with invariant mass...





Planning round 4
FeatureAgent `four\_jet\_feature\_agent` built 37 ML features for 75,126 selected events.
PlanningAgent proposed `exotic\_higgs\_hist\_gradient\_boosting`: a `hist\_gradient\_boosting` strategy using 8 features. Rationale: Try hist_gradient_boosting with invariant masses, angular separations, and b-tag multiplicity to classify exotic-Higgs signal events from QCD heavy-flavour background events because hist_gradient_boosting often performs better on tabular data.
ClassificationAgent `exotic\_higgs\_hist\_gradient\_boosting` trained `hist\_gradient\_boosting` using 8 features. Validation AUC = 0.753; test AUC = 0.750. Hyperparameters: {'learning_rate': 0.1, 'max_iter': 150, 'max_depth': None, 'max_leaf_nodes': 31, 'min_samples_leaf': 20}.
SignalAnalysisAgent found the best threshold at score >= 0.20, with $S/\sqrt{B}$ = 25.02.

	round		strategy	model_family	features_used	hyperparameters	test_auc	best_threshold	best_S_over_sqrt_B	rationale
0	1		physics_mass_random_forest	random_forest	13	{"max_depth": null, "max_features": "sqrt", "m...	0.741719	0.2	24.234381	Use invariant masses, angular separations, and...
1	2		exotic_higgs_classifier	random_forest	20	{"max_depth": null, "max_features": "sqrt", "m...	0.817933	0.5	41.955002	Try to classify exotic-Higgs signal events fro...
2	3		physics_mass_hist_gradient_boosting	hist_gradient_boosting	6	{"learning_rate": 0.1, "max_depth": null, "max...	0.718856	0.2	21.563611	Try hist_gradient_boosting with invariant mass...
3	4		exotic_higgs_hist_gradient_boosting	hist_gradient_boosting	8	{"learning_rate": 0.1, "max_depth": null, "max...	0.749879	0.2	25.018312	Try hist_gradient_boosting with invariant mass...



Final Baseline versus Best Planned Model Comparison

This final comparison selects the planned model with the best threshold-scan value of $S/\sqrt{B}$, then compares it with the baseline random-forest workflow.

```
In [68]: # Ensure the planning loop has been run and planned_results is not empty
if not planned_results:
    raise RuntimeError("Run the Agentic Planning Loop before running this final comparison cell.")

# Select the best planned result based on the highest S/sqrt(B) value
best_result = max(planned_results, key=lambda result: result["best"]["S_over_sqrt_B"])
best_agent = best_result["agent"] # The ClassificationAgent instance for the best result
best_plan = best_result["plan"] # The plan dictionary for the best result
best_scan = best_result["scan"] # The threshold scan table for the best result
best_scores = best_result["scores"] # The prediction scores for the best result

print("Baseline model:")
# Print description of the baseline classifier
print(baseline_classifier.describe())
# Print the best S/sqrt(B) and threshold for the baseline
print(f"Baseline best S/sqrt(B) = {baseline_best['S_over_sqrt_B']:.2f} at threshold {baseline_best['threshold']:.2f}.")
print()

print("Best planned model:")
# Print description of the best planned classifier
print(best_agent.describe())
# Print the best S/sqrt(B) and threshold for the best planned model
print(
    f"Best planned S/sqrt(B) = {best_result['best']['S_over_sqrt_B']:.2f} "
```

```
        f"at threshold {best_result['best']['threshold']:.2f}."
    )
    # Print the planning rationale, features used, and hyperparameters for the best planned model
    print(f"Planning rationale: {best_plan['rationale']}")
    print(f"Features used ({len(best_plan['feature_columns'])}): {'', '.join(best_plan['feature_columns'])}")
    print(f"Hyperparameters: {json.dumps(best_plan['hyperparameters'], sort_keys=True)}")

    # Create a DataFrame for easy comparison of baseline and best planned model metrics
    comparison = pd.DataFrame([
        {
            "model": "baseline",
            "strategy": baseline_classifier.name,
            "model_family": baseline_classifier.strategy["model_family"],
            "features_used": len(baseline_classifier.feature_names_),
            "test_auc": baseline_classifier.metrics["test"]["auc"],
            "best_threshold": baseline_best["threshold"],
            "best_S_over_sqrt_B": baseline_best["S_over_sqrt_B"],
        },
        {
            "model": "best_planned",
            "strategy": best_plan["strategy_name"],
            "model_family": best_plan["model_family"],
            "features_used": len(best_plan["feature_columns"]),
            "test_auc": best_agent.metrics["test"]["auc"],
            "best_threshold": best_result["best"]["threshold"],
            "best_S_over_sqrt_B": best_result["best"]["S_over_sqrt_B"],
        },
    ])
    # Display the comparison table
    display(comparison)

    # Calculate ROC curve data for the baseline classifier
    baseline_fpr, baseline_tpr, _ = roc_curve(baseline_classifier.y_test, baseline_scores)
    # Calculate ROC curve data for the best planned classifier
    best_fpr, best_tpr, _ = roc_curve(best_agent.y_test, best_scores)

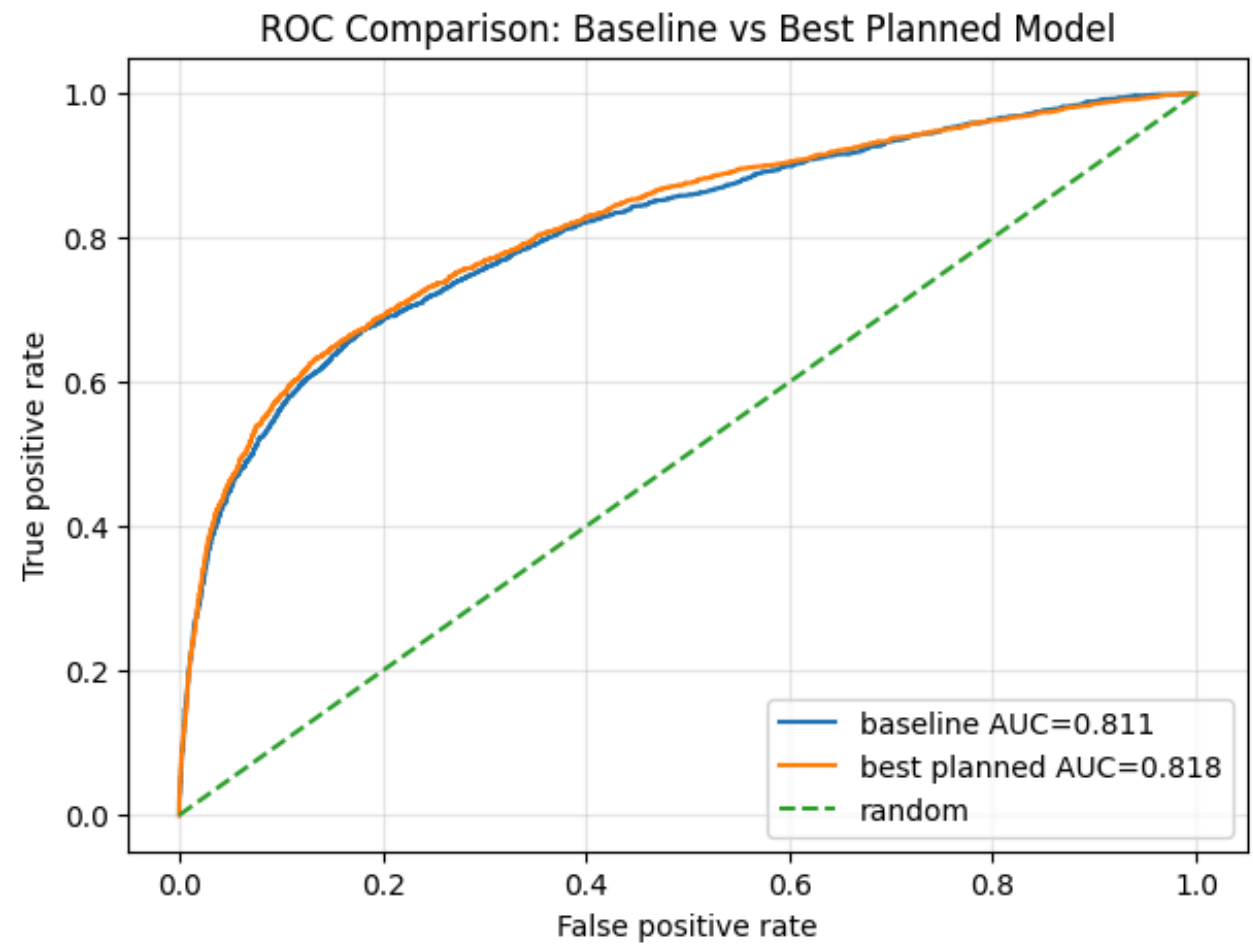
    plt.figure(figsize=(7, 5)) # Create a new figure for ROC comparison
    # Plot ROC curve for baseline
    plt.plot(baseline_fpr, baseline_tpr, label=f"baseline AUC={baseline_classifier.metrics['test']['auc']:.3f}")
    # Plot ROC curve for best planned model
    plt.plot(best_fpr, best_tpr, label=f"best planned AUC={best_agent.metrics['test']['auc']:.3f}")
    # Plot random classifier line
    plt.plot([0, 1], [0, 1], linestyle="--", label="random")
    plt.xlabel("False positive rate") # Set x-axis label
    plt.ylabel("True positive rate") # Set y-axis label
    plt.title("ROC Comparison: Baseline vs Best Planned Model") # Set title
    plt.legend() # Display legend
    plt.grid(True, alpha=0.3) # Add grid
    plt.show() # Show plot

    plt.figure(figsize=(7, 5)) # Create a new figure for S/sqrt(B) comparison
    # Plot S/sqrt(B) for baseline
    plt.plot(baseline_scan["threshold"], baseline_scan["S_over_sqrt_B"], marker="o", label="baseline")
    # Plot S/sqrt(B) for best planned model
    plt.plot(best_scan["threshold"], best_scan["S_over_sqrt_B"], marker="o", label="best planned")
    plt.xlabel("Classifier threshold") # Set x-axis label
    plt.ylabel("S / sqrt(B)") # Set y-axis label
    plt.title("Signal-Sensitivity Curve: Baseline vs Best Planned Model") # Set title
    plt.legend() # Display legend
    plt.grid(True, alpha=0.3) # Add grid
    plt.show() # Show plot
```

Baseline model:
ClassificationAgent `baseline\_random\_forest` trained `random\_forest` using 37 features. Validation AUC = 0.811; test AUC = 0.811. Hyperparameters: {'n_estimators': 160, 'min_samples_leaf': 2, 'max_depth': None, 'max_features': 'sqrt'}.
Baseline best S/sqrt(B) = 39.87 at threshold 0.40.

Best planned model:
ClassificationAgent `exotic\_higgs\_classifier` trained `random\_forest` using 20 features. Validation AUC = 0.814; test AUC = 0.818. Hyperparameters: {'n_estimators': 180, 'min_sample_s_leaf': 2, 'max_depth': None, 'max_features': 'sqrt'}.
Best planned S/sqrt(B) = 41.96 at threshold 0.50.
Planning rationale: Try to classify exotic-Higgs signal events from QCD heavy-flavour background events using invariant masses, angular separations, and b-tag multiplicity.
Features used (20): ht, m_4j, m_jj_12, deltaR_jj_12, m_jj_13, deltaR_jj_13, m_jj_14, deltaR_jj_14, m_jj_23, deltaR_jj_23, m_jj_24, deltaR_jj_24, m_jj_34, deltaR_jj_34, min_deltaR_jj, n_btags, jet1_btag, jet2_btag, jet3_btag, jet4_btag
Hyperparameters: {"max_depth": null, "max_features": "sqrt", "min_samples_leaf": 2, "n_estimators": 180}

	model	strategy	model_family	features_used	test_auc	best_threshold	best_S_over_sqrt_B
0	baseline	baseline_random_forest	random_forest	37	0.811072	0.4	39.867926
1	best_planned	exotic_higgs_classifier	random_forest	20	0.817933	0.5	41.955002



Signal-Sensitivity Curve: Baseline vs Best Planned Model

